

Suitably Impressive Thesis Title



Candidate Number: 1234567

University of Oxford

Word Count: 15000

A thesis submitted for the degree of

Master of Science in Advanced Computer Science

Trinity 2026

Abstract

Your abstract goes here. Check your departmental regulations, but this is usually expected to be under 300 words — roughly one page.

A workable structure is one paragraph per move: (1) context and why the problem matters, (2) the gap, (3) what you did, (4) what you found, and (5) what it means more broadly.

Acknowledgements

Thank your supervisors first, then anyone whose ideas or feedback shaped the direction of the work.

Then colleagues, collaborators, and friends.

Then any funding or institutional support you received.

Finally, family.

Contents

List of Figures

Chapter 1

Introduction

1.1 Motivation

1.2 Contributions

1.3 Thesis Overview

Chapter 2

Background

2.1 Generative Modelling

The Goal of Generative Modelling

Given that the focus of the project is on generative modelling with flow maps for inverse problem solving, we first formalise the goal of generative modelling.

Let us say we want to generate examples of a particular class of object. We can specify this class to be as broad or as narrow as we would like. For example 2D images, 2D images of cats, or molecular structures. Importantly all objects within the class must be representable as a vector in the same continuous Euclidean space R^D . For example, images of height H , width W and 3 colour channels can be represented as a vector of dimensionality $D = H \times W \times 3$, after flattening. While certain classes of objects of discrete nature, for example text and graph data, cannot naturally be represented as a vectors in a continuous space R^D , we limit our focus to generative modelling for objects that can be, in this project.

For illustration, let us choose the class of objects we want to generate to be images of cats. In the space of R^D , some regions of the space will contain vectors corresponding to images of cats, while other regions do not. In fact, the manifold hypothesis states that a class of objects form a manifold in R^D . *In machine learning, we assume there exists an underlying unknown probability distribution over R^D* which places more density in regions of space which contain vectors corresponding to images that look more like a cat. More generally, given an arbitrary object class in R^D , we assume there exists an underlying probability distribution p_{data} over R^D which concentrates its mass on those vectors that are plausible instances

of the class, and assigns negligible mass elsewhere

Under this view, the primary objective of generative modelling is, once specifying a particular class of object, and typically given a finite dataset of N samples of that object class:

$$\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \quad \text{where} \quad \mathbf{x}_i \sim p_{\text{data}}(\mathbf{x})$$

to learn a generative model which can generate new, synthetic samples which are distributed according to the true data distribution for that object class. More generally, generative modelling aims to produce samples from any given target probability distribution, p_{target} .

2.1.1 Reward Alignment

In many cases, we do not simply want to learn a generative model which can generate examples of a particular class of object, $s \sim p_{\text{data}}$, but we want to generate examples of that class of object with certain 'desirable properties'. What precisely is meant by desirable properties can be specified by a *reward function*, $r(\mathbf{x})$. An example of a reward function for images is one which computes an aesthetic rating. So we would like to build a generative model which can generate samples from p_{data} for which $r(\mathbf{x})$ is large. This task, which we call reward alignment, can be formalised as sampling from a reward-tilted distribution, which reweights the density of the data distribution towards regions which have high reward according to the specified reward function

$$\tilde{p}_{\text{data}}^{\text{tilt}}(x) \propto p_{\text{data}}(x) \exp(r(x)) \quad (2.1)$$

This problem of reward alignment encapsulates many settings, such as conditional generation (i.e. generating an object conditioned on some data y , such as a textual prompt or class) and inverse problem solving.

In this project, we focus on inverse problem solving as it is a particularly difficult reward alignment setting. However, our work generalises to any reward alignment setting. In the related work section, we discuss the main ideas used to tackle general reward alignment, with the awareness that all such methods can be applied to the specific inverse problem solving setting.

We now introduce the goal of inverse problem solving and discuss its connection to reward alignment.

2.1.2 Inverse Problems

To define an inverse problem, we must first define the forward problem. Given a clean object $x \sim p_{data} \in \mathbb{R}^d$ (for example an image), a 'forward-operator' A acts on it to produce an 'observation' y :

$$y = A(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

We also add a random Gaussian noise to produce the final observation. Examples of how a forward-operator A might act on an image include: blurring, inpainting (deleting a region of pixels) or downsampling. The 'forward problem' is computing the observation y from a clean object x .

The inverse problem is the reverse. Given observation y , recover the clean object x from which it came from. Since the forward operator A is typically destroys information (and is hence inverible), there can be several distinct clean objects which map to the same observation y under the forward-operator A . Since we cannot recover x exactly, our goal is instead to generate objects x distributed according to p_{data} which are consistent with the given observation y . We can formalise this goal as sampling from $p(x|y) = \frac{p(y|x)p_{data}(x)}{p(y)}$. Here we expanded the conditional distribution using Bayes' rule. The $p(y|x)$ term is called the likelihood. Since $y = A(x) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$, it follows that

$$p(y | x) = \mathcal{N}(y; A(x), \sigma^2 I) \propto \exp\left(-\frac{\|y - A(x)\|^2}{2\sigma^2}\right)$$

We can now rewrite the posterior as

$$p(x | y) \propto p(x) \exp\left(-\frac{\|y - A(x)\|^2}{2\sigma^2}\right) \quad (2.2)$$

Thus x has high probability under the conditional distribution if both 1) it is consistent with the observation (i.e. $A(x)$ is close to y) and 2) it is a plausible object according to the data distribution, p_{data} . The goal of inverse problem solving is given an observation y , i.e. samples from p_{data} , which are consistent with the observation y .

Relation to Reward Alignment

We now dicuss how inverse problem solving can be seen as a special case of reward alignment.

Recall that in inverse problem solving, for a given measurement y , we wish

to sample from the conditional distribution $p(x|y)$. Comparing with the form of a general reward-tilted distribution $p(x|y) \propto p(x) \exp(r(x, y))$, it is clear that, for a fixed measurement y , the inverse problem posterior takes exactly the form of the reward-tilted distribution $p(x|y)$, with $r(x) = -\frac{\|y - A(x)\|^2}{2\sigma^2} = \log p(y|x) + \text{const.}$ A high reward hence corresponds to a small residual $\|y - A(x)\|^2$.

Note that, in practice, we do not want to learn to sample from the reward tilted distribution $\tilde{p}_{\text{data}}^{\text{tilt}}(x)$ only for a fixed measurement y . Instead, for any measurement y , we want to learn to sample from

$$\tilde{p}_{\text{data}}^{\text{tilt}}(x|y) \propto p_{\text{data}}(x) \exp(r(x, y)) = p(x|y) \quad (2.3)$$

where $r(x, y) = -\frac{\|y - A(x)\|^2}{2\sigma^2}$. In other words, we wish to learn to sample from a whole family of tilted distributions $\tilde{p}_{\text{data}}^{\text{tilt}}(x|y)$.

2.2 Flow-based generative models

The goal of the project is to tackle reward alignment, in particular inverse problems, rather than purely unconditional generation. However, many approaches for reward alignment build upon unconditional generative modelling methods. Given that flow-based generative modelling has emerged as the dominant paradigm for generative modelling for several data modalities, achieving state-of-the-art results in domains such as image and video generation, we seek to tackle reward alignment using flow-based models. Most recently, flow maps has emerged as a method which only requires one neural network evaluation for generation (rather than the several required by the previous normalising flow and flow map models). However, adapting unconditional flow maps for general reward alignment problems, while maintaining only one neural network evaluation has largely been untackled, with Variational Flow Maps, being the only work in this line. While variational flow maps requires fine-tuning an unconditional flow map to tackle reward alignment, the goal of our project is to extend the approach of VFM to not require fine-tuning.

Given that our project ultimately builds on unconditional flow-based generative models, review essential background on flow-based generative models for unconditional generation - keeping in mind that the goal of the project is ultimately to tackle general reward alignment.

2.2.1 Normalising Flows

The goal set out in Section ?? is to construct a generative model, i.e. an algorithm which can generate samples from a given target distribution. Normalising flows is one such algorithm.

A normalising flow models an intractable target distribution by pushing a simple base distribution $p_{\text{prior}}(z)$, for which we *can* sample and evaluate the density of, through a learned map. More precisely, given a sample from a tractable base distribution $X_1 \sim p_{\text{prior}}$ we wish to learn a map $f_\phi : \mathbb{R}^D \rightarrow \mathbb{R}^D$ (parameterised by a neural network with parameters ϕ) such that $X_0 = f_\phi(X_1)$ is distributed according to the target distribution i.e. $X_0 \sim p_{\text{target}}$. We denote the distribution obtained by pushing $p_{\text{prior}}(z)$ through f_ϕ in this way as p_ϕ . The task is then to learn parameters ϕ such that p_ϕ approximates p_{target} well. How ϕ is learned is the subject of Section ??.

The defining property of normalising flows is that the learned map f_ϕ must be invertible and both it and its inverse is continuously differentiable. A map that satisfies these properties is called a diffeomorphism. Now because f_ϕ is a diffeomorphism, the change of variables formula ?? allows us to evaluate the density of p_ϕ at any x

$$p_\phi(x) = p_{\text{prior}}(f_\phi^{-1}(x)) \left| \det \frac{\partial f_\phi^{-1}(x)}{\partial x} \right| \quad (2.4)$$

Or equivalently, (given formulation which evaluates f_ϕ rather than f_ϕ^{-1})

Normalising flows therefore enables both 1) sampling of p_ϕ and 2) evaluation of its density. These two operations require different computational requirements. Sampling requires being able to sample from the base distribution, and evaluating the forward direction of the learned map f_ϕ . Density evaluation (according to ??) requires computing the inverse direction of the learned map f_ϕ^{-1} and the determinant of the Jacobian of either f_ϕ or f_ϕ^{-1} , and the density of the base distribution.

The predominant difficulty in density evaluation is the computation of the determinant of a $D \times D$ Jacobian matrix $\frac{\partial f_\phi^{-1}(x)}{\partial x}$. While we can always compute the Jacobian (and subsequently its determinant) of a neural network with D inputs and D outputs using D passes of either forward or reverse mode automatic differentiation, computing the determinant of a general $D \times D$ matrix costs $O(D^3)$ operations, which is very expensive.

Much research has therefore gone into different ways to construct f_ϕ so that its Jacobian has special structure which makes computing its determinant

cheap. Of course, when constructing f_{phi} , we still must ensure that it is a diffeomorphism, and is flexible enough to transport our base distribution to the chosen target distribution.

Constructing Normalising Flows

We have stated that f_{phi} is parameterised by a neural network, however we require that f_{phi} is invertible, which is not necessarily true of an arbitrary neural network. Furthermore, we would also like for the determinant of the Jacobian of f_{phi} to be computed cheaply, while still ensuring that it *is flexible enough to model the chosen target distribution* – the – art normalising flow architectures that ensure that f_{phi} has these properties that we want.

The first principle is to break f_{phi} down into a composition of L simpler invertible maps, which we typically refer to as layers.

$$f_{phi} = f_{L-1} \circ f_{L-2} \circ \dots \circ f_0$$

A sample from p_{prior} is transported to a sample from p_{phi} by applying each layer in sequence

$$x_{k+1} = f_k(x_k), \quad k = 0, \dots, L-1, \quad x_0 = z \sim p_{prior}, \quad x_L = x \sim p_{phi} \quad (2.5)$$

Importantly, the composition of invertible maps remains invertible, so we can still use the change of variables formula to evaluate the density of p_{phi} . The Jacobian log determinant is given by (28 in NF survey) The advantage of this approach is that increasing L results in only $O(L)$ increase in operations required to compute the log determinant. Now, if we strict the functional form of each map f_k so that its Jacobian has special structure which makes computing its determinant cheaply, then the overall

We now discuss how to construct an individual f_k , which must of course be a diffeomorphism. Recall of – the – art methodssuch as Glow and Tarnflow are a variant of autoregressive flows.

To simplify notation, let us drop the layer index k for a moment and consider a single layer mapping an input vector $z \in \mathbb{R}^D$ to an output vector $z' \in \mathbb{R}^D$. In an autoregressive flow, we split the coordinates of z into K equal parts. There are two main special cases, when $K=D$, we have fully autoregressive flows and when $K=2$, this is known as using ‘coupling layers’. Let us denote these parts as $z_1, \dots, z_K$. We transform each part z_i , one at a time, from $i = 1$ to K , in order to iteratively compute z' part by part. The transformation of z_i is performed elementwise $\tau(z_i, h_i)$, where $h_i = c_i(z_{<i})$. Here τ , which we call the transform (not to be confused with transform

th' conditioner', which is typically implemented as a neural network, and only takes as input parts $z_{<i}$. In

To see why this construction is invertible, consider any z' . We can prove by induction that we can compute each part z'_i iteratively from $i = 1$ to K . We can compute z'_1 as follows : h_1 can be computed automatically as it is not a function of any coordinates of z , then since we can invert the $i = 1$ conditioner, then we can compute z'_1 as follows : compute h_1 , which is a function of $z_1, \dots, z_{i-1}$, then invert the transformer.

Note that computing f_{θ} in the forward direction, each h_i (and thus each z'_i) can be computed in parallel, $1, \dots, K$. Hence inverse computation is slower and more expensive than forward computation. This is ultimately

So we have shown that we can evaluate the inverse of autoregressive flows. We now show that autoregressive flows have cheap Jacobian determinant. In particular, since the j -th coordinate in each block z'_{ij} , is computed as a transformation of z_{ij} , and the parameter determinant of the entire layer can be computed in $O(D)$ time.

Now, previously, we stated that the transformer is an elementwise monotone function, and that the conditioner is typically implemented as a neural network. However, there are several possible options for how exactly to implement both the transformer and conditioner. Importantly, depending on the target distribution, we must ensure that both are sufficiently flexible enough to represent a map which transforms the base distribution to the target. For the transformer, popular choices include implementing it as an affine transformation or a monotonic spline. We discuss affine transformation in more detail in the next section. For the conditioner, in theory we could implement the i -th conditioner as its own separate neural network. However, as previously discussed, since in forward direction, we can compute all h_i in parallel (assuming the conditioner takes as input z i.e. MAF direction), it would be more ideal to have

Above, we have discussed a general recipe for constructing autoregressive flows. Several works in the literature instantiate this recipe in different ways. For example, RealNVP, Glow, MAF, NSF. However, in this project, we use the TarFlow architecture, since it has shown to be state of the art for modelling distributions over images, and is a highly scalable architecture. We now discuss exactly the design choices that TarFlow makes to instantiate this general autoregressive flow recipe.

Recall that in autoregressive flows we break z down into K blocks. But we never mentioned exactly how. TarFlow is for learning a target distribution which is over images z' . And so z' corresponds to a flattened image. Given this, what is a sensible way to break z down into blocks? Consider a $C \times H \times W$ image where C is channels, H is height, W is width. Split the H and W dimensions into a grid of

$PATCH_{SIZE} \times PATCH_{SIZE}$ patches. Each patch contains $PATCH_{SIZE} \times PATCH_{SIZE} \times C$ numbers. Now have a sequence of $K = (H/PATCH_{SIZE}) \times (W/PATCH_{SIZE})$ patches. Each patch

The next choice that TarFlow makes is the transformer. It chooses an affine transformer. In particular, τ is an affine transformation

$z'_i = \tau(z_i, h_i) = \alpha_i * z_i + \beta_i$, where $h_i = (\alpha_i, \beta_i)$ and $*$ denotes element-wise product. so with in each token, each coordinate is transformed via its own affine map. As previously

For the conditioner, TarFlow natively chooses the MAF orientation. i.e. conditioner takes as input coordinates of z rather than z' . Furthermore, for computational efficiency, TarFlow implements the conditioner as a neural network which can take all of z as input and output all of $(h_1, \dots, h_K)$ simultaneously. And since the conditioner is such a $PATCH_{SIZE} \times C$ to dimension CH . It then adds learned positional embeddings to each projected token. This is so the parameters applied to tokens are computed from tokens strictly before it, and token 0 gets (0, 0).

Let's zoom into the transformer layer. Each transformer layer consists of a multi-headed attention layer, followed by MLP. this is just standard attention: for each position, we compute keys, queries and values, take dot product between keys and queries to compute scores, set scores for which query index key index to -inf, then for each query, take softmax across scores for that query, to get final attention scores, add linear combination of value vectors to each position, where weights of linear combination are determined by the attention scores.

We just now discussed a single autoregressive layer f_k . TarFlow uses num_blocks layers. This is a hypothesis.

Everything we have discussed so far is the architecture of TarFlow. We now discuss a non-architectural trick that TarFlow uses to aid training the normalising flow to match an target distribution over images. Note that real images occupy a 'thin' sliver of $\mathbb{R}^D$. i.e. a manifold. Remember that a flow is a smooth bijection with smooth inverse. the base

the only problem is that now we are modelling a noisy distribution fortunately, given a noisy image, we can recover a clean image using Tweedie's formula but basically you add to the noisy image the score and the score is the gradient of the log density and log density is exactly what we can evaluate using a normalising flow

Training Normalising Flows

So far, we have introduced the framework of normalising flows: in order to sample from some intractable distribution p_{target} , learn a diffeomorphism f_ϕ that pushes p_{prior} to some distribution $p_\phi \approx p_{\text{target}}$. We have also discussed autoregressive flows as a specific construction of normalising flows, which has properties that we care about. However, we have yet to discuss how we learn the parameters ϕ

of the neural network f_ϕ , such that p_ϕ indeed approximates p_{target} well. That is the goal of this section.

Since our ultimate goal is for p_ϕ to be close to p_{target} , a natural objective to minimise is the KL divergence between both distributions. Let us recall the definition of KL divergence. Consider any distribution p which represents the true distribution, and q which represents the model distribution you are training to approximate p . The forward KL divergence between p and q is defined as

$$D_{\text{KL}}(p \parallel q) = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right]$$

whereas the reverse KL divergence is $D_{\text{KL}}(q \parallel p)$. We must distinguish between forward and reverse KL because KL divergence is not symmetric in its arguments. Thus we can minimise either the reverse or forward KL divergence between p_{target} and p_ϕ and we now discuss the consequences of both choices.

Forward KL divergence

Expanding out the definition of forward KL divergence, we have

$$\begin{aligned} \mathcal{L}(\phi) &= D_{\text{KL}}[p_{\text{target}} \parallel p_\phi] \\ &= \mathbb{E}_{x \sim p_{\text{target}}} \left[\log \frac{p_{\text{target}}(x)}{p_\phi(x)} \right] \\ &= -\mathbb{E}_{x \sim p_{\text{target}}} [\log p_\phi(x)] + \mathbb{E}_{x \sim p_{\text{target}}} [\log p_{\text{target}}(x)] \\ &= -\mathbb{E}_{x \sim p_{\text{target}}} [\log p_\phi(x)] + \text{const} \end{aligned} \tag{2.6}$$

where $\mathbb{E}_{x \sim p_{\text{target}}} [\log p_{\text{target}}(x)]$ is a constant as it is independent of ϕ . Substituting the expression for $\log p_\phi(x)$?? into ?? and estimating the expectation with Monte Carlo samples $\{x_n\}_{n=1}^N$ drawn from p_{target} , we have

$$\mathcal{L}(\phi) \approx -\frac{1}{N} \sum_{n=1}^N \left[\log p_{\text{prior}}(f_\phi^{-1}(x_n)) + \log \left| \det \frac{\partial f_\phi^{-1}(x_n)}{\partial x} \right| \right] + \text{const}$$

Here, when estimating the expectation with Monte Carlo samples we see that minimising forward KL is precisely maximum likelihood estimation. Further note that evaluating $\mathcal{L}(\phi)$ requires computing f_ϕ^{-1} and the determinant of its Jacobian. Importantly, it never requires f_ϕ during training nor does it require evaluating the density of p_{target} .

In forward KL, since the expectation in ?? is taken over p_{target} , if p_ϕ assigns

little density at any point that p_{target} has mass, then at that point $-\log p_\phi \rightarrow \infty$ and so the penalty explodes. In other words, in forward KL, p_ϕ is forced to cover every mode of p_{target} even at the cost of placing mass unnecessarily between modes of p_{target} .

Reverse KL divergence

Expanding out the definition of reverse KL divergence, we have

$$\mathcal{L}(\phi) = D_{\text{KL}}[p_\phi \parallel p_{\text{target}}] = \mathbb{E}_{x \sim p_\phi} [\log p_\phi(x) - \log p_{\text{target}}(x)] \quad (2.7)$$

Unlike in forward KL divergence, we now must evaluate the density of p_{target} . However, it is enough to evaluate an unnormalised density, $\tilde{p}_{\text{target}}(x)$, where $p_{\text{target}}(x) = \tilde{p}_{\text{target}}(x)/C$ with C unknown. Since C is independent of ϕ , it vanishes when computing the gradient of ?? during gradient-based optimisation.

Further note that in gradient-based optimisation, when taking the derivative of ?? with respect to ϕ , we cannot pass the derivative through the expectation, as the expectation is taken with respect to p_ϕ . To pass the derivative through the expectation (which subsequently allows us to compute an unbiased Monte Carlo estimator of ??), we can use the reparameterisation trick. In particular, we rewrite the expectation over $z \sim p_{\text{prior}}$ rather than $z \sim p_\phi$ and use the fact that $x = f_\phi(z)$

$$\begin{aligned} \mathcal{L}(\phi) &= \mathbb{E}_{z \sim p_{\text{prior}}} [\log p_\phi(f_\phi(z)) - \log p_{\text{target}}(f_\phi(z))] \\ &= \mathbb{E}_{z \sim p_{\text{prior}}} \left[\log p_{\text{prior}}(z) - \log \left| \det \frac{\partial f_\phi(z)}{\partial z} \right| - \log p_{\text{target}}(f_\phi(z)) \right] \end{aligned} \quad (2.8)$$

In the final equality, we use the change of variables formula ?? to allow us to evaluate the density of p_ϕ at $f_\phi(z)$. Since we already know z , we do not have to evaluate f_ϕ^{-1} .

In reverse KL, since the expectation in ?? is over p_ϕ , the integrand is only evaluated where p_ϕ has placed mass. Hence reverse KL does not penalise not placing mass at regions where p_{target} does place mass (as forward KL does). Instead reverse KL penalises placing mass where p_{target} *does not* place mass. We say that reverse KL is mode-seeking.

2.2.2 Flow Matching

In this section, we give an overview of flow models and the predominant algorithm for training them, flow matching. This section provides my understanding of the course material from MIT's An Introduction to Flow Matching and Diffusion Models.

Recall that Normalising flows transform noise samples from a tractable base distribution p_{base} to samples from a target distribution p_{target} via a finite chain of invertible maps.

Flow models instead model the transformation between noise and data samples as a continuous-time transformation over a given time interval. The interval is typically taken to be $t \in [0, 1]$ where t denotes the time variable. We assume that at $t = 1$, we begin with a sample $x_1 \sim p_{\text{base}}$ from the base distribution, and as t decreases, the sample is continuously evolved until at time $t = 0$, we obtain a sample x_0 which is distributed according to the target distribution. The advantage of this continuous approach is that it does not restrict us to learning the entire transformation between noise and data in one big jump (or finite series of discrete jumps as in Normalising Flows).

The defining characteristic of a flow model is that the dynamics of this continuous-time transformation between noise and data is governed by an Ordinary Differential Equation (ODE). Contrast this with Diffusion models, which instead choose the continuous-time transformation to be governed by a Stochastic Differential Equation (SDE).

ODE Definition

In order to define an ODE, we must first define both a trajectory and a vector field. A trajectory represents the path a single particle takes through space, across time. Formally, it is a function of time t , and assigns each time a location in space $\mathbb{R}^D$

$$X : [0, 1] \rightarrow \mathbb{R}^d, \quad t \mapsto X_t$$

A vector field specifies how to move in space (as described by a velocity vector), at any location and time. Formally, it is a function of the form

$$u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, \quad (x, t) \mapsto u_t(x)$$

An ODE is an equation which constrains a trajectory to move according to a given vector field. It is additionally paired with an initial condition which specifies where the trajectory must start. Mathematically, both constraints can be written as

$$\frac{d}{dt}X_t = u_t(X_t) \quad X_0 = x_0$$

In words, the instantaneous rate of change (derivative) of the particle’s position with respect to time must equal the vector field evaluated at the particle’s current position.

A solution to an ODE is simply a trajectory X_t (for $t \in [0, 1]$) which satisfies both constraints. We may now wonder, for any given ODE and initial condition, does a solution always exist, and if so, is it unique? It can in fact be proven (Picard-Lindelof Theorem) that if the vector field $u_t(x)$ of the ODE is continuously differentiable and its derivatives are bounded, then the ODE has a unique solution.

Fortunately, neural networks with standard activation functions (like SiLU or GELU) are continuously differentiable and have bounded derivatives. Therefore, for our case of interest, where we define ODEs using a vector field parameterised by a neural network, we can safely assume that all ODEs have a unique solution.

Flow Model Definition

At the beginning of this section, we stated that flow models model the transformation between noise and data samples as a continuous-time transformation, whose dynamics are governed by an ODE. Having defined ODEs, we are now in a position to make this idea concrete. Given a sample from a tractable base distribution $X_1 \sim p_{\text{prior}}(z)$, a flow model specifies its trajectory X_t (for $t \in [0, 1]$) as governed by the following ODE

$$\frac{d}{dt}X_t = u_t^\theta(X_t) \tag{2.9}$$

Here, u_t^θ is a vector field, which is parameterised by a neural network with parameters θ . We denote the distribution over trajectory locations at end-time $t = 0$ as p_θ , so that $X_0 \sim p_\theta$. Note that the randomness over the trajectory end-points comes from the initial random starting point X_0 , rather than the ODE itself, which is deterministic given an initial starting point. The task is to learn the parameters θ such that p_θ approximates p_{target} well.

Sampling from a Flow Model

Before we discuss how we train the velocity network u_t^θ of a flow model, we discuss how to sample from a flow model. In particular, assuming that we have successfully trained u_t^θ , how do we determine the trajectories specified by the ODE ??, so that we can ultimately generate novel samples $X_0 \sim p_\theta$.

Because the vector field u_t^θ is parameterised by a neural network, we typically cannot analytically solve for a trajectory which satisfies the ODE. We must therefore approximate the trajectory using a numerical method, of which Euler’s method is one of the simplest. In Euler’s method, we initialise a starting point of the trajectory $X_1 = x_1$ and unroll the trajectory iteratively via

$$X_{t-h} = X_t - hu_t(X_t) \quad (t = 1, 1-h, 1-2h, \dots h) \quad (2.10)$$

where $h = n^{-1} > 0$ is referred to as the step size and $n \in \mathbb{N}$ is the number of simulation steps. Intuitively, Euler’s method proposes that to move from X_t to X_{t+h} along the trajectory, simply look at the velocity at the current position $u_t(X_t)$, and take a step in that direction, with step size proportional to the duration h . It is clear that the smaller the step size, the more accurate the simulated trajectory is. Because in a flow model, the time convention is such that we move backwards in time from $t = 1$ to 0, we subtract rather than add the velocity in ??.

Flow Matching

Recall that flow models generate samples by sampling from a base distribution $X_1 \sim p_{\text{base}}$, and subsequently simulating an ODE ?? defined by a learned vector field. The endpoint X_0 at $t = 0$ of the simulated trajectory is a generated sample. As we discussed, our goal is to train the neural network u_t^θ such that X_0 is distributed according to the target distribution. Flow matching is an efficient, state-of-the-art, training algorithm to achieve this goal. While the flow matching algorithm itself is very simple, its derivation is fairly involved. We give an overview of the logical construction of the derivation here.

To summarise this logical construction, recall our goal requires desired ODE trajectory fulfills $X_1 \sim p_{\text{init}}, X_0 \sim p_{\text{target}}$. The goal does not restrict how the intermediate trajectory p_{base} at $t = 1$, and $p_0 = p_{\text{target}}$ at $t = 0$. Given that we have explicitly constructed a probability path, we can for a given target sample $z \sim p_{\text{target}}$, a conditional probability path is a sequence of conditional distributions $p_t(z_t = z | z_0 = z)$. Note that there are many possible choices which can satisfy the definition of

a conditional probability path. We discuss a popular choice at the end of this section.

if we marginalise over a conditional probability path, we get the marginal probability path. we obtain the density for marginal probability path using sum rule and product rule.

while the density is intractable as it involves computing an integral over all possible z in $\mathbb{R}^D$, *we can sample from a marginal probability path, as long as it is possible to sample from the*

importantly, it can be proved that the marginal probability path interpolates between base distribution and the target distribution.

We can now specify the goal of flow matching more precisely. We want to learn a vector field such that, for each time t in $[0,1]$, the point along the trajectory X_t is distributed according to the probability path p_t .

We now explicitly construct exactly such a vector field (called the marginal vector field), which is the target vector field that flow matching aims to learn.

We must first define the conditional vector field. For any given sample z from the target distribution, and given a particular conditional probability path, the conditional vector field $u_t^{\text{target}}(z)$ must be such that if a trajectory X_t satisfies the ODE defined by condition $u_t(X_t|z)$ then that trajectory is distributed according to the conditional probability path, i.e. $X_t \sim p_t(z)$.

It can be proven that the marginal vector field follows the marginal probability path. More precisely, If $X_0 \sim p_0$ and $d/dt X_t = u_t(X_t)$ then the trajectory X_t is distributed according to p_t .

So now we can specify the goal of flow matching further. We want to train the neural network u_t^f such that it is equal to precisely the marginal vector field. One idea is to simply regress

However, the conditional vector field is tractable as we can derive an analytical formula for it, and so we instead regress against this (26) This is called the conditional flow matching loss. It can in fact be proved that the conditional flow matching loss equals the flow matching loss up to a constant. Therefore their gradients coincide and the minimiser of the conditional loss is precisely equal to the minimiser of the flow matching loss.

So we have learned that flow matching training consists of learning the marginal vector field. However to construct a marginal vector field, we must choose a valid conditional probability path. We give a concrete example of a very simple conditional probability path and use it to derive a conditional vector field and subsequently a corresponding conditional flow matching loss. This conditional probability path, called the the CondOT probability path is used to train state-of-the-art flow models, and is the basis for the training objectives used to train the generative models used in our

project. The CondOT probability path is defined as $p_t(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{t}\mathbf{z}, (1-t)\mathbf{I})$, often referred to as the CondOT probability path (make sure it satisfies the time convention). Clearly it satisfies the definition of a conditional probability path since at $t=1$, $p_1(\mathbf{x}|\mathbf{z})$ is distributed according to standard normal and at $t=0$, it is distributed according to a Dirac at $\mathbf{z}$. While there are many possible conditional vector fields, we can take $\mathbf{v}_t(\mathbf{x}) = \mathbf{z} - \mathbf{x}$, where $\mathbf{x} \sim \mathcal{N}(0, \mathbf{I})$, we can take the expectation over standard normal to give (31).

For the CondOT conditional probability path, we now summarise the flow matching training algorithm. We sample a target from the dataset, we noise that target, $\mathbf{t}\mathbf{z} + (1-t)\mathbf{e}$, (this is a linear interpolation between noise and data), and we simply regress the velocity network evaluated at the noised point against $(\mathbf{z} - \mathbf{e})$, which is simply the difference between noise and data. We are effectively regressing against a straight line.

2.2.3 Flow Maps

When sampling from a flow model, we start at random noise $\mathbf{X}_1$ (simple), and simulate the trajectory $\mathbf{x}_t$ (for $t \in [0, 1]$) by taking small steps in the direction output by the velocity network, $\mathbf{u}_t(\mathbf{x}_t)$. Often a large number of steps are taken.

The displacement resulting from following the vector field $\mathbf{v}$ from $t=s$ to $t=t$ (for any s and t , and taking an infinite number of infinitesimally small steps) is exactly an integral of $\mathbf{v}$

$$\int_s^t \mathbf{v}(\mathbf{x}_\tau, \tau) d\tau$$

By taking small steps in the direction output by $\mathbf{u}_t(\mathbf{x}_t)$ (as in Euler's method) we are effectively approximating $\mathbf{x}_t = \mathbf{x}_s + \int_s^t \mathbf{v}(\mathbf{x}_\tau, \tau) d\tau$. (2.11) Given a source time s , location on a trajectory at that time s , and a target time t , a flow map outputs the location on that same trajectory at time t . Hence assuming that we have successfully trained a neural network to predict the flow map, we can sample $\mathbf{X}_1$, and sample evaluate $F(\mathbf{X}_1, 1, 0)$ in order to obtain the sample $\mathbf{X}_0$ at $t=0$. Hence sampling from a flow model only requires one forward pass of a neural network.

We now take some time to convince ourselves that a flow map is indeed a well-defined function. Since we ask the flow map to take as input a source time s , and location on a trajectory at that time s , and correspondingly output the location on that same trajectory at time t , we implicitly assume that 1) there is only one such trajectory such that at time s , we are at location $\mathbf{x}_s$, so that there is no ambiguity regarding which trajectory we are on, (in other words, a location and time s deterministically lead to a single location $\mathbf{x}_t$, so that the flow map always has a single valid output.

To understand why points 1 and 2 are true, recall that trajectories are ultimately specified by the learned vector field. In particular, if we sample a start

point X_1 , and follow the vector field, until $t = 0$, we obtain a trajectory. Suppose 1) was not true, and that 0 , and coverge at x_s , then we still have one valid velocity only, so this is fine? it is only if we go backwards in

Now that we defined what a flow map is (in relation to a given marginal vector field, or equivalently, in relation to a given conditional probability path), and shown that it is indeed a well-defined function, we now discuss how to train a neural network to learn a flow map. Training a flow map is significantly harder than training a flow model as the former provides a global characterisation of the trajectories between noise and data while the latter only provides a local characterisation.

Many different algorithms have been proposed to train flow maps, and we refer readers to Sander for an intuitive overview of the different approaches and how they relate. Here, we provide some intuition on how to derive the MeanFlow Training algorithm, that is used to train the flow maps used in this project. We choose MeanFlow because it is shown to be state of the art (cite results), and importantly enables training a flow map from scratch, without using a pre-trained velocity network as a starting point for training.

The MeanFlow algorithm is derived from a simple intuitive rule which any valid flow map must satisfy. Given any flow map, consider the instantaneous change in the output of $F(\mathbf{x}_s, s, t)$ as we change the time of the current location, s . Intuitively, changing the current time on a given trajectory, will have no effect on the location at time t on the trajectory. In particular, for every valid $\mathbf{x}_s, s$ and t (assuming a CondOT probability path, valid x_s means it is a sample from the marginal probability), $\frac{d}{ds}F(\mathbf{x}_s, s, t) = 0$. This is the Eulerian consistency rule. While this rule seems trivial, it can be proved that any function

Since s affects two inputs of F , we can expand the equation using via chain rule as

$$\frac{d}{ds}F(\mathbf{x}_s, s, t) = \nabla_{\mathbf{x}_s}F(\mathbf{x}_s, s, t)\frac{d\mathbf{x}_s}{ds} + \frac{\partial}{\partial s}F(\mathbf{x}_s, s, t) = 0 \quad (2.12)$$

If we use a loss function, which is simply the magnitude of the left hand side (in practice we take the square of the left hand side), and averaged over all $\mathbf{x}_s, s$ and t (?), the neural network parameterising F which minimises this loss function perfectly to 0

$$\mathcal{L}_{\text{Eulerian}} = \mathbb{E} \left[\left(\frac{\partial}{\partial s}F(\mathbf{x}_s, s, t) + \nabla_{\mathbf{x}_s}F(\mathbf{x}_s, s, t)\frac{d\mathbf{x}_s}{ds} \right)^2 \right] \quad (2.13)$$

Here the expectation is taken over $\mathbf{x}_s, s$ and t , where s and t are distributed according to uniform $[0, 1]$, and

In practice, can we evaluate all the terms in this loss function? We note that $\frac{d\mathbf{x}_s}{ds} = v(\mathbf{x}_s, s)$. We must ensure that $v(\mathbf{x}_s, s)$ corresponds to the velocity of the marginal vector field (for a specified conditional probability path). This can be

achieved by first training a flow model and using that as a reference for $v(\mathbf{x}_s, s)$. This approach is called distillation as we are effectively distilling a flow model into a flow map. However, we will soon discuss how the MeanFlow algorithm avoids this, and instead trains a flow map from scratch.

For the remaining terms in the loss, we see that they are derivative of exactly the function, F , that we are trying to learn. Because F is parameterised by a neural network, we can indeed compute these derivatives of F using backpropagation. However, when optimising the loss function using gradient descent, we must then compute higher order derivatives of the neural network parameterising F . This can be expensive. Fortunately, there are a couple tricks that MeanFlow uses to avoid this, which we now show.

First, note that rather than parameterising a flow map $F(\mathbf{x}_s, s, t)$ to directly predict the location $\mathbf{x}_t$ on $\frac{1}{t-s} \int_s^t v(\mathbf{x}_\tau, \tau) d\tau$, we say that these are equivalent parameterisations of a flow map because we can reconstruct $\mathbf{x}_s + (t-s)V(\mathbf{x}_s, s, t)$

Going back to the Eulerian consistency equation ??, if we substitute F for the mean flow parameterisation V , we obtain

$$\frac{\partial}{\partial s} (\mathbf{x}_s + (t-s)V(\mathbf{x}_s, s, t)) + \nabla_{\mathbf{x}_s} (\mathbf{x}_s + (t-s)V(\mathbf{x}_s, s, t)) v(\mathbf{x}_s, s) = 0.$$

Now, using product rule, we can simplify this as

$$-V(\mathbf{x}_s, s, t) + (t-s) \frac{\partial}{\partial s} V(\mathbf{x}_s, s, t) + v(\mathbf{x}_s, s) + (t-s) \nabla_{\mathbf{x}_s} V(\mathbf{x}_s, s, t) v(\mathbf{x}_s, s) = 0. \quad (2.14)$$

We rearrange to isolate V

$$V(\mathbf{x}_s, s, t) = v(\mathbf{x}_s, s) + (t-s) \left(\frac{\partial}{\partial s} V(\mathbf{x}_s, s, t) + \nabla_{\mathbf{x}_s} V(\mathbf{x}_s, s, t) v(\mathbf{x}_s, s) \right).$$

Intuitively, this equation states that the average velocity between time s and t equals to the instantaneous velocity at time s plus some correction term which involves the derivative of V .

We can think of the right hand side as the target, which we would like V to match. However, if we simply form the loss function by squaring in ??), then this RHS target still involves computing derivatives of exactly the function V we are trying to optimise. To avoid this, we can wrap the target in a 'stop-gradient'

operation. This blocks gradient flow during backpropagation.

$$\mathcal{L}_{\text{MeanFlow}} = \mathbb{E} \left[\left(V(\mathbf{x}_s, s, t) - \text{sg} \left[v(\mathbf{x}_s, s) + (t - s) \left(\frac{\partial}{\partial s} V(\mathbf{x}_s, s, t) + \nabla_{\mathbf{x}_s} V(\mathbf{x}_s, s, t) v(\mathbf{x}_s, s) \right) \right] \right)^2 \right]. \quad (2.15)$$

Now during optimisation, we basically assume the target is a constant, and only optimise V so that it matches this constant target. Note that the gradient direction calculated by computing this loss function is not equal to the gradient direction of the true loss function (without the stop gradient) that we wish to optimise. However, in practice, this is fine. An additional trick that MeanFlow uses to stabilise training is to compute the target using an exponential moving average (EMA) of V 's parameters. Otherwise, the target is constantly moving as V is updated.

Now we have tackled the problem of avoiding computing higher order derivatives when optimising the loss function, we return to the problem of needing to evaluate the marginal vector field, v , in order to compute the loss function. To avoid requiring a pre-trained flow model to provide these velocities, MeanFlow uses the exact same marginalisation trick used to derive the training objective for flow matching. (Is this definitely the marginalisation trick? I thought the marginalisation trick states that marginal vector field follows the marginal probability path? but the result here states we can replace marginal vector field with conditional vector field in the loss. this is theorem 12 in the MIT notes?)

In particular, we can simply replace the marginal vector field, v , in the training objective with the conditional vector field. When using the CondOT conditional probability path, the conditional vector field $p_t(x_0)$ is exactly $\varepsilon - x_0$ (sampled from standard normal). Hence the training objective becomes $\mathcal{L}_{\text{MeanFlow}} = \mathbb{E} \left[\left(V(\mathbf{x}_s, s, t) - \text{sg} \left[(\varepsilon - \mathbf{x}_0) + (t - s) \left(\frac{\partial}{\partial s} V(\mathbf{x}_s, s, t) + \nabla_{\mathbf{x}_s} V(\mathbf{x}_s, s, t) (\varepsilon - \mathbf{x}_0) \right) \right] \right)^2 \right]. \quad (2.16)$

The gradient of this loss function is exactly the gradient of $??$, and they also have the same minimiser. We do not prove this result here. However, we note that the proof only works because of the design choices that MeanFlow makes. In particular, using mean squared error as the loss, the fact that we have used the Eulerian consistency rule (rather than alternative consistency rules) and the fact that we used the stop-gradient operation. The stop gradient thus both avoids higher-order differentiation and also allows us to use the marginalisation trick (again is this the marginalisation trick? maybe best to call it marginal from conditional learning).

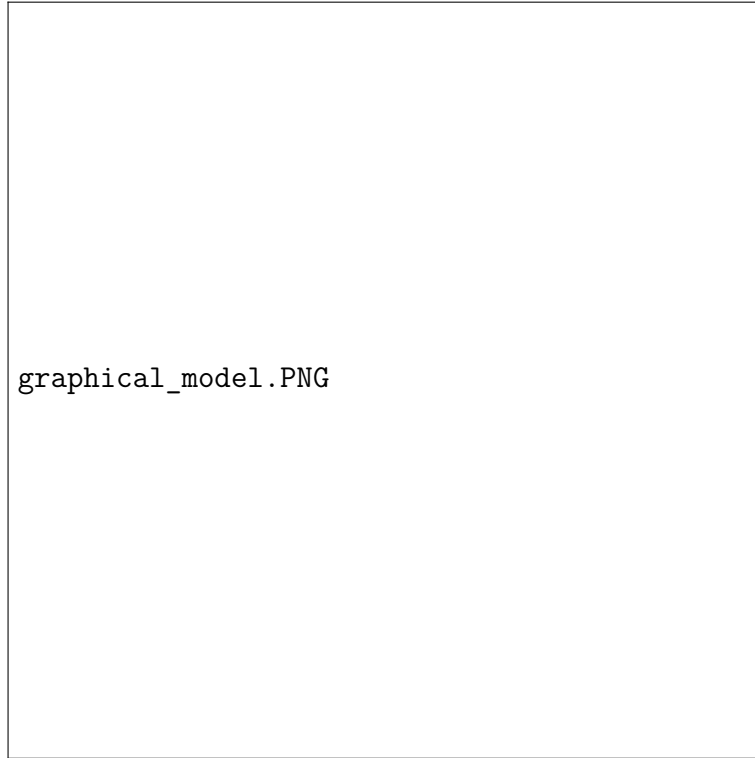


Figure 2.1: Left: Graphical model with unknown stochastic latent variables z , observed variables x , and deterministic parameters θ . Right: Graphical model with stochastic per-datapoint local latent variables z_n , observed variables x_n , and deterministic global parameters θ .

2.3 Variational Inference

We have now discussed various unconditional flow-based generative models which enable sampling from a data distribution p_{data} . Recall that our ultimate goal is inverse problem solving.

In brief, given a distribution $p(z) = p(z)/C$ which we can evaluate the density of up to a normalising constant C , but can't sample from, the goal of variational inference is to find an approximation of this distribution called a 'variational distribution' which we can sample from and evaluate the exact density of.

Bayesian inference, where we want to sample from a posterior distribution, $p(z | y) = p(y | z)p(z)/C$, is a common setting in which we want to use variational inference. We will soon see that variational flow maps formulates inverse problem solving with flow maps as a bayesian posterior inference problem. We therefore now make this bayesian posterior inference problem precise, and discuss how variational inference can be used to solve it.

Consider a probabilistic model with unknown (latent) variables z , known variables x , and fixed known parameters θ , as shown in the left of Figure ???. In

Bayesian posterior inference, we want to sample from and evaluate the density of the posterior distribution $p_\theta(z | x)$. Both of these operations are not possible because it requires computing the normalising constant $p_\theta(x)$ which is usually an integral or sum over the entire latent variable space

$$p_\theta(z | x) = \frac{p_\theta(x, z)}{p_\theta(x)}, \quad p_\theta(x) = \int p_\theta(x, z) dz.$$

Since we can't evaluate the density or sample from the true posterior, variational inference chooses a tractable family of distributions $q_\psi(z)$ (which we can evaluate the density of and sample from) and optimises over ψ so that $q_\psi(z)$ is as close as possible to the true posterior $p_\theta(z | x)$. The variational parameters ψ parameterises the chosen tractable family of distributions (for example if the family is chosen to be Gaussian, ψ consists of mean and variance). We mathematically measure the 'closeness' between our approximation $q_\psi(z)$ and true posterior $p_\theta(z | x)$ by KL divergence

$$D_{\text{KL}}(q_\psi(z) \parallel p_\theta(z | x)) \tag{2.17}$$

The goal of variational inference is to minimise this KL divergence with respect to ψ . Let us expand this objective function ??:

$$\begin{aligned} D_{\text{KL}}(q_\psi(z) \parallel p_\theta(z | x)) &= \mathbb{E}_{q_\psi(z)} \left[\log \frac{q_\psi(z)}{p_\theta(z | x)} \right] \\ &= \mathbb{E}_{q_\psi(z)} \left[\log q_\psi(z) - \log \left(\frac{p_\theta(x | z)p_\theta(z)}{p_\theta(x)} \right) \right] \\ &= \underbrace{\mathbb{E}_{q_\psi(z)} [\log q_\psi(z) - \log p_\theta(x | z) - \log p_\theta(z)]}_{\mathcal{L}(\psi | \theta, x)} + \log p_\theta(x) \end{aligned}$$

The first equality follows from the definition of KL divergence. The second equality follows from log rules and Bayes' law. In the final equality, we can pull $\log p_\theta(x)$ out of the expectation because it is a deterministic constant with respect to z .

Additionally, when finding ψ that minimises D_{KL} , we can ignore $\log p_\theta(x)$ because it is a constant with respect to ψ . Thus minimising D_{KL} is equivalent to minimising $\mathcal{L}(\psi | \theta, x)$. In variational inference, we define the negative of this quantity, the ELBO, and seek to maximise this

$$\tilde{\mathcal{L}}(\psi | \theta, x) = \mathbb{E}_{q_\psi} [\log p_\theta(x, z) - \log q_\psi(z)].$$

It is common to rewrite the ELBO as

$$\begin{aligned}
\tilde{\mathcal{L}}(\psi \mid \theta, x) &= \mathbb{E}_{q_\psi(z)}[\log p_\theta(x, z) - \log q_\psi(z)] \\
&= \mathbb{E}_{q_\psi(z)}[\log p_\theta(x \mid z) + \log p_\theta(z) - \log q_\psi(z)] \\
&= \mathbb{E}_{q_\psi(z)}[\log p_\theta(x \mid z)] - D_{\text{KL}}(q_\psi(z) \parallel p_\theta(z)) \tag{2.18}
\end{aligned}$$

We can hence interpret the ELBO as the sum of two distinct terms: expected log likelihood and negative KL from posterior to prior.

2.3.1 Amortised VI

We now consider the graphical model shown in the right of Figure ???. Given this new graphical model, the generative model is of the form

$$p(\mathcal{D}, z_{1:N} \mid \theta) = \prod_{n=1}^N p(x_n, z_n \mid \theta)$$

Here, $\mathcal{D} = x_{1:N}$ denotes the dataset (observed variables). The equality follows from the fact that each observed variables x_n only depends on z_n , and that each x_n is independent and identically distributed. Under this new model, we now wish to approximate N posteriors, one for each latent variable z_n for $n = 1 \dots N$

$$p_\theta(z_n \mid x_n) = \frac{p_\theta(x_n, z_n)}{p_\theta(x_n)} \tag{2.19}$$

We approximate each posterior $p_\theta(z_n \mid x_n)$ with a separate variational distribution $q_{\psi_n}(z_n)$. As in the previous section, we optimise each ψ_n by maximising the corresponding ELBO. In particular, we wish to maximise

$$\tilde{\mathcal{L}}_n(\psi_n \mid \theta, x_n) = \mathbb{E}_{q_{\psi_n}(z_n)}[\log p_\theta(x_n \mid z_n)] - D_{\text{KL}}(q_{\psi_n}(z_n) \parallel p_\theta(z_n))$$

with respect to ψ_n for each $n = 1, \dots, N$.

In practice, if N is large, it is very costly to optimise ψ_n separately for each new observed variable x_n . To avoid this, amortised variational inference, instead seeks to learn a model $f_\phi(x_n)$ which can output the variational parameters ψ_n , given any observed variable x_n as input. This is called amortised VI because we spread (amortise) the cost of heavy per-example optimisation of each ψ_n across the entire dataset, by training a single shared network, $f_\phi(x_n)$. In amortized VI, we now represent the approximate posterior as $q_\phi(z_n \mid x_n)$ to reflect that

the approximation no longer depends on local variational parameters ψ_n , but instead global variational parameters ϕ , and the data point x_n . We sum the per-datapoint ELBOs to obtain the following overall objective function, which we seek to maximise with respect to ϕ

$$\tilde{\mathcal{L}}(\theta, \phi \mid \mathcal{D}) = \sum_{n=1}^N \left[\mathbb{E}_{q_\phi(z_n|x_n)} [\log p_\theta(x_n, z_n) - \log q_\phi(z_n \mid x_n)] \right] \quad (2.20)$$

2.3.2 Stochastic VI

To avoid computing the loss function over the entire dataset x_n from $n = 1, \dots, N$, we can compute an unbiased estimate of this by drawing a random batch B of B datapoints from the

2.4 Variational Flow Maps

Recall that our goal is to solve inverse problems with flow maps. This project extends the approach taken by Variational Flow maps. Given a pre-trained unconditional flow map that can generate samples from $p_{data}(x)$, *VFM is a method which fine-tunes this flow map*

Say we have already trained a flow map which can generate samples from $p_{theta}(x)$, *which closely approximates $p_{data}(x)$. Now, in inverse problems solving, assuming a particu*

Now, recall we previously discussed that a flow map (or equivalently the marginal vector field) induces a bijection between samples from the base distribution and samples from the target distribution. In particular, it can be proved that $F(\cdot, 1, 0)$ is a diffeomorphism between noise and data samples. Hence the change of variables formula tells us that applying F^{-1} to the distribution $p_{theta}(x|y)$ gives us a well-defined distribution in noise space $p_{theta}(z|y)$.

If we can learn to sample from the distribution, $p_{theta}(z|y)$, these samples can be decoded by the flow

How do we know that such noise that decodes onto samples from $p_{theta}(x|y)$ exists? Well, samples

Goal of VFM is to sample from this posterior, $p_{theta}(z|y) \cdot \tilde{p}_{noise}^{tilt}(z \mid y) \propto p(z) \exp\left(-\frac{\|y - A(f_\theta(z))\|^2}{2\sigma^2}\right)$, $p(z) = \mathcal{N}(0, I)$ (2.21) Since we can evaluate the density of this posterior up to a normalising constant, variational inference is the natural solution. Recall that variational inference seeks to approximate a distribution, which we can evaluate up to a normalising constant, with a variational distribution that we can sample from. Furthermore, note that sampling from the posterior $p_{theta}(z|y)$ is exactly a Bayesian posterior inference problem. Hence we can apply the derivations from

The graphical model describes the underlying process by which we assume the data y_n (corresponding to observations) is generated. we assume that for a given observation y_n (observe

$A * f_{\theta}(z_n) + e$. The flow map parameters, θ , capture the process by which data is generated. Note that $f_{\theta}(z_n)$ is a deterministic function of z_n , we ignore it in the graphical model, and do not include an edge.

This graphical model is exactly the same as which we saw in the previous section (and is also the graphical model that a VAE assumes). Hence, we can use variational inference to learn for each $p_{\theta}(z_n | y_n)$, a variational distribution $q_{\phi}(z_n)$ approximating it. (relabelled as y_n .)

$$\tilde{\mathcal{L}}(\theta, \phi | \mathcal{D}) = \sum_{n=1}^N \left[\mathbb{E}_{q_{\phi}(z_n | y_n)} [\log p_{\theta}(y_n, z_n) - \log q_{\phi}(z_n | y_n)] \right] \quad (2.22)$$

$$\begin{aligned} &= \sum_{n=1}^N \left[\mathbb{E}_{q_{\phi}(z_n | y_n)} [\log p_{\theta}(y_n | z_n) + \log p_{\theta}(z_n) - \log q_{\phi}(z_n | y_n)] \right] \\ &= \sum_{n=1}^N \left[\mathbb{E}_{q_{\phi}(z_n | y_n)} [\log p_{\theta}(y_n | z_n)] - D_{\text{KL}}(q_{\phi}(z_n | y_n) \parallel p_{\theta}(z_n)) \right] \end{aligned} \quad (2.23)$$

Here, we rearrange the objective to reveal two terms with distinct roles. Maximising the observation fit term $\mathbb{E}_{q_{\phi}}[\log p_{\theta}(y_n | z_n)] = \mathbb{E}_{q_{\phi}} \left[\frac{\|y_n - A(f_{\theta}(z_n))\|^2}{2\sigma^2} \right] + \text{const}$ pulls the mass of the variational distribution $q_{\phi}(z_n | y_n)$ towards noise points z_n for which $A(f_{\theta}(z_n))$ matches the observation y_n . On the other hand, the KL term pulls the variational distribution close to the noise prior $\mathcal{N}(0, I)$. This is important because we know that the true distribution $\tilde{p}_{\text{noise}}^{\text{tilt}}(z | y)$ is a reshuffling of $\mathcal{N}(0, I)$ in theory.

The tractable family of distributions that the variational distribution $q_{\phi}(z_n | x_n)$ parameterises can in theory be chosen as any family of distributions, as long as we can compute ??.

VFM choose a simple diagonal Gaussian. For a diagonal Gaussian, we can safely compute ??.. In particular, we can sample from it (so that we can estimate the expectation), and we can also analytically evaluate the KL divergence to $p_{\theta}(z_n)$ because this prior too is a gaussian.

Given an observation y_n , the shared network g_{ϕ} outputs both the mean and diagonal values of the covariance matrix $\Sigma_{\phi}(y_n)$.

$$g_{\phi}(y) = (\mu_{\phi}(y), \sigma_{\phi}^2(y)), \quad q_{\phi}(z | y) = \mathcal{N}(z | \mu_{\phi}(y), \text{diag}(\sigma_{\phi}^2(y)))$$

This neural network is what the authors of VFM call the 'noise adapter' (and is otherwise known as an encoder, in the context of a VAE).

2.4.1 Joint training of noise adapter and flow map

In practice, a simple diagonal gaussian is not expressive enough to approximate the true posterior $p_\theta(z|y)$ accurately. The authors of VFM theoretically prove that expectation of decomposition

To overcome this, the authors of VFM propose fine-tuning the flow map jointly with the noise adapter. Using the same graphical model ??, rather than assuming parameters of the flow map, θ , are fixed and known, we now seek to update them. For this, we can do maximum likelihood estimation. In particular, we wish to find θ which maximises $\log p(\mathcal{D} | \theta) = \sum_{n=1}^N \log p(x_n | \theta)$. Note that to compute the log likelihood of for a single data point x_n , we must marginalise over the unknown latent variable z_n . Hence $p(x_n | \theta)$ is intractable. However, note that the ELBO is a lower bound on $p(x_n | \theta)$. This is clear to see since,

$$\log p_\theta(x_n) = D_{\text{KL}}(q_\psi(z_n) \parallel p_\theta(z_n | x_n)) + \tilde{\mathcal{L}}_n(\psi_n, \theta | x_n) \quad (2.24)$$

and $D_{\text{KL}} > 0$ by definition. Hence in order to maximise $\log p_\theta(x_n)$, we can instead maximise the lower bound surrogate $\tilde{\mathcal{L}}_n(\psi_n, \theta | x_n)$ with respect to θ , keeping ψ fixed. In order to maximise $\log p(\mathcal{D} | \theta)$, the full objective, is thus to maximise $\sum_{n=1}^N \tilde{\mathcal{L}}_n(\psi_n, \theta | x_n)$ with respect to θ . This is exactly the same objective ?? which we used to optimise the variational parameters. Hence we can learn the noise adapter parameters and flow map parameters jointly, by taking derivative with respect to both sets of parameters, and doing a gradient descent update on both sets simultaneously. It is also common to alternatively maximising the ELBO objective with respect to ψ (holding θ fixed) and vice versa.

Now, the only problem with this objective is that it does not impose any consistency properties of flow maps, such as the Eulerian rule. This means that during training the flow map might degrade into a function which no longer is a valid flow map. By valid flow map, we mean that it encodes the solution trajectories of an ODE defined by marginal vector field. To overcome this, VFM adds MeanFlow loss function to the ELBO objective, scaled by a coefficient which represents how strongly to enforce the Eulerian consistency rule.

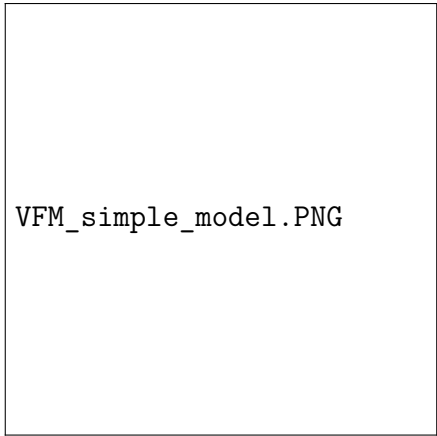
$$\mathcal{L}_{\text{VFM}}(\phi, \theta) = \mathcal{L}_{\text{MF}} + \mathcal{L}_{\text{obs}} + \mathcal{L}_{\text{KL}} \quad (2.25)$$

The MeanFlow term is

$$\mathcal{L}_{\text{MF}} = \frac{1}{2\tau^2} \mathbb{E} \left\| V_\theta(x_s, s, t) - \text{sg} \left[(\epsilon - x_0) - (s - t) \frac{d}{ds} V_\theta \right] \right\|^2$$

This is the same loss function used to pre-train the flow map. This term lets the flow map change without forgetting how to generate the original data distribution. The role of τ is to determine how strongly the flow map changes to minimise the MeanFlow term versus the observation term.

The authors of VFM derive this objective in a principled way. However, the focus of this project is not joint training, but instead to keep the flow map frozen, so we omit the full proof here, and refer readers to the VFM paper. To summarise their proof, in the current graphical model, the clean data x has been marginalised out: we only have $z \rightarrow y$. To train the model to generate real data, x , not just data which produces the measurement y under the forward operator, we need to introduce the clean data into the graphical model. This is to simultaneously align the data variable, observation y and latent noise variable, not just the observation and latent noise variable. We define a generative joint and inference joint, and minimise the KL divergence between these. From this we get the objective.



VFM_simple_model.PNG

Figure 2.2: Enter Caption

Chapter 3

Related Work

Recall that the goal of this project is reward alignment, i.e. sampling from a reward tilted distribution

$$\tilde{p}_{\text{data}}^{\text{tilt}}(x) \propto p_{\text{data}}(x) \exp(r(x)) \quad (3.1)$$

Many methods exist for this, which either train a generative model from scratch, fine-tune an unconditional generator, or which assumes access to an unconditional generator but leaves it fixed (and hence is training-free with respect to the generator). Given that in our project, our method keeps the unconditional generator fixed, in this section, we focus on methods which also take this approach, so that we have a fair comparison. Hence we do not discuss methods such as vanilla guidance, classifier free guidance, or guidance-free-training which train a conditional flow model or flow map from scratch or by fine-tuning. We also ensure that the methods we compare against do not assume that the reward is fixed in advance, since in inverse problems, each new observation defines a new reward.

Rather than learning to sample from the noise distribution which the marginal vector field transports to the reward-tilted distribution (as is the philosophy of VFM), the underlying idea behind the majority of reward alignment approaches in the literature is based on nudging the unconditional marginal vector field output to follow an alternative vector field which transports the standard normal directly to the reward tilted data distribution. During the sampling procedure of a flow model, these methods estimate this 'nudge' term and add it to the unconditional velocity network's output, before taking the next ODE step. Such methods are hence training-free with respect to the unconditional generative

model. Note it is also possible to fine-tune the velocity network to directly match this nudged vector field, however as previously mentioned, we only consider methods utilise a fixed unconditional generator. All such approaches are known as guidance-based methods, as we are effectively guiding trajectories to end at samples from reward-tilted distribution. We now make formalise the framework of this approach, and discuss the different ways that this approach is implemented in practice.

3.1 The Reward-tilted vector field

Given a reward tilted data distribution $\tilde{p}_{\text{data}}^{\text{tilt}}(x) \propto p_{\text{data}}(x) \exp(r(x))$, and an unconditional marginal vector field u_t (defined with respect to CondOT-based probability path which starts standard normal, and $p_0 = p_{\text{data}}$), it can be proved that the following 'guided' vector field $u_t^r(x_t) = u_t(x_t) + a_t \nabla_{x_t} V_t(x_t)$ (3.2) follows a probability path which starts at $p_1 = p_{\text{data}}$ and ends at $p_0 = p_{\text{tilt}}$. Here, assuming that u_t is marginal vector field built on CondOT conditional probability path, $a_t = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \beta_t \dot{\beta}_t\right)$ (we need to change this for CondOT rather than general gaussian probability path), and V_t is called the value function.

We now take some time to define and understand the value function. The value function is defined as

$$V_t(x_t) = \log \mathbb{E}_{z \sim p_{1|t}(\cdot|x_t)}[\exp(r(z))] \quad (3.3)$$

Note that the reward function is defined such that it is only evaluable on clean data. Therefore, the value function measures the expected **future** reward that you will obtain if you continue along the trajectory from your current noisy state x_t . Going back to the guided vector field ??, we can hence interpret it as nudging the unconditional output of the marginal vector field toward region of space for which the expected reward is higher.

A careful reader may note that if you deterministically follow the learned marginal vector field u_t^θ from x_t , you will always end at the same clean sample z , and so given this, expected reward is deterministic. However, the expectation is taken with respect to

$$p_{1|t}(z|x_t) = \frac{p_t(x_t|z)p_{\text{data}}(z)}{p_t(x_t)}$$

which represents the distribution of all possible clean samples z such that sampling from the corresponding conditional probability path, $p_t(x_t|z) = \mathcal{N}(x_t; \alpha_t z, \beta_t^2 I)$, produces x_t . In other words, $p_{1|t}(z|x_t)$ represents the distribution of clean

endpoints z which are consistent with x_t according to the conditional probability path $p_t(x_t|z)$.

In order to sample from the reward-tilted distribution, during sampling procedure, you estimate $a_t \nabla_{x_t} V_t(x_t)$ on the fly and manually add it to the model’s output before taking your next ODE step. We now discuss different approaches to estimate the gradient of the value function. The main difficulty in estimating the value function is due to the fact that we cannot easily sample from $p_{1|t}(\cdot|x_t)$.

3.2 Estimating the Value Function with Flow models

Make the reward function robust to noise.

Rather than approximating V_t directly, we can adapt the reward function r so that it can be evaluated directly on noise x_t . This circumvents having to sample clean points from $p_{1|t}(\cdot|x_t)$. Classifier guidance is a popular example takes exactly this approach. In classifier guidance, we train a time dependent classifier, which can be interpreted as a reward function, that can be evaluated on noisy inputs. A limitation of this approach is that it is difficult to learn a neural network which can score noisy inputs.

Approximating the posterior as the mean

Several methods approximate the value function by computing the reward at the mean of the posterior. In particular we make the following approximation

$$V_t(x_t) = \log \mathbb{E}_{p_{1|t}}[\exp(r(z))] \approx \log[\exp(r(\mathbb{E}_{p_{1|t}}[z]))] = r(\mathbb{E}_{p_{1|t}}[z])$$

. This approximation requires us to evaluate the mean of the posterior, $\mathbb{E}[z | x_t] = \mathbb{E}_{p_{1|t}}[z]$. Tweedie’s formula gives us exactly this, and we now derive it. First recall that, the marginal vector field is defined as

$$u_t(x_t) = \int u_t(x_t | z) \frac{p_t(x_t | z) p_{\text{data}}(z)}{p_t(x_t)} dz = \mathbb{E}_{z \sim p_{1|t}(\cdot|x_t)}[u_t(x_t | z)].$$

Second, note that, assuming CondOT conditional probability path, we note that given clean data point z , if we have $x_t \text{simp}_t(z)$, then $x_t = tz + (1 - t)\varepsilon$ and

$$u_t(x_t | z) = z - \varepsilon.$$

Noting that $\varepsilon = \frac{x_t - tz}{1-t}$, we determine that

$$u_t(x_t | z) = z - \frac{x_t - tz}{1-t} = \frac{(1-t)z - x_t + tz}{1-t} = \frac{z - x_t}{1-t}.$$

Substitute into the marginal field and pull the expectation through

$$u_t(x_t) = \mathbb{E}_{z \sim p_{1|t}(\cdot | x_t)} \left[\frac{z - x_t}{1-t} \right] = \frac{\mathbb{E}_{p_{1|t}}[z] - x_t}{1-t}.$$

Rearranging gives

$$\mathbb{E}_{z \sim p_{1|t}(\cdot | x_t)}[z] = x_t + (1-t)u_t(x_t)$$

That is Tweedie’s formula for condOT. This gives us a cheap way to approximate $V_t(x_t) \approx r(\mathbb{E}_{p_{1|t}}[z])$ as it only requires one forward pass of the neural network, u_t^θ . Furthermore, we can backpropagate through u_t^θ in order to compute the gradient of the value function approximation $\nabla_{x_t} r(\hat{z}(x_t))$ (where $\hat{z}(x_t) := \mathbb{E}_{p_{1|t}}[z | x_t] = x_t + (1-t)u_t^\theta(x_t)$).

Several methods (DPS, PSLD, MPGD, FlowChef, FlowDPS) for inverse problem solving take this approach of approximating the mean as the posterior (using reward function as $r(x) = \log(p | x) + \text{const}$). The main difference between these approaches is whether they compute the gradient $\nabla_{x_t} r(\hat{z}(x_t))$ exactly (DPS, PSLD), or whether they approximate it to avoid backpropagating through u_t^θ (MPGD, FlowChef, FlowDPS). Once computing the gradient direction, the methods also differ in the coefficient applied to the direction when adding it to the output of the unconditional velocity network. Such coefficient is never exactly a_t , *and instead is often time – dependent, and/or tuned*.

Several inverse problem solvers (DPS, PSLD, MPGD, FlowChef, FlowDPS) take this approach of approximating the posterior by its mean, with reward function $r(x) = \log p(y | x) + \text{const}$. The main difference between these approaches is whether they compute the gradient $\nabla_{x_t} r(\hat{z}(x_t))$ exactly (DPS, PSLD), or approximate it by $\nabla_{\hat{z}} r(\hat{z})$ to avoid backpropagating through u_t^θ (MPGD, FlowChef, FlowDPS). Having computed the gradient direction, the methods also differ in the coefficient applied to it when adding it to the output of the unconditional velocity network. This coefficient is never exactly a_t , and is often tuned empirically.

The downside to this method is that you are collapsing the whole posterior, $p_{1|t}(z|x_t)$ to its *mean* before applying the reward. Especially when t is small (x_t is very noisy), $p_{1|t}(z|x_t)$ will be very broad, and so collapsing it to its mean

before computing the reward function may produce a poor approximation of the true value function. Furthermore, the mean of the posterior distribution may not even correspond to a valid sample (i.e. it does not lie on the manifold of the data distribution). It is likely a blurry interpolation between modes of the the posterior distribution. Evaluating the reward function on this blurry interpolation may not produce a meaningful score.

Monte Carlo approximation of the posterior

We can estimate the value function more honestly via a Monte Carlo approximation. In particular, we draw N clean samples $z^{(i)} \sim p_{1|t}(\cdot|x_t)$ and compute the Monte Carlo approximation of the value function

$$V_t(x_t) \approx \log \frac{1}{N} \sum_{i=1}^N \exp(r(z^{(i)})) \quad (3.4)$$

You might be wondering how exactly we draw posterior samples $z^{(i)}$. Earlier we discussed that if we follow the learned ODE, u_t^θ , from x_t , we will deterministically land on exactly one clean sample z . Therefore, in order to draw diverse set of posterior samples, each sample must be drawn by simulating the SDE from x_t . Fortunately, if we have a pre-trained flow model, u_t^θ , it is easy enough to use this to simulate the SDE:

$$dX_t = \left(u_t(X_t) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t) \right) dt + \sigma_t dW_t$$

Remember that the score function $\nabla \log p_t(X_t)$ can be recovered from u_t^θ via the known linear relationship that we previously derived (when using a Gaussian probability path).

Going back to ??, we now have an unbiased estimator of the true value function. This is a much better approximation of the value function. The downside of this approach is the cost required to compute ??. To draw even one sample $z^{(i)}$, you must simulate the SDE from x_t to x_1 , which requires many forward passes of the neural network, u_t^θ . Furthermore, to compute $\nabla_{x_t} V_t(x_t)$, we now have to backpropagate through the entire multi-step sampling chain, and for many samples $z^{(i)}$. This is memory-prohibitive.

Sequential Monte Carlo

So far we have been trying to control trajectories by nudging the marginal vector field by an estimate of $\nabla_{x_t} V_t(x_t)$. An alternative approach is to simulate many trajectories in parallel via an SDE, and at each time step, and for each trajectory, compute the mean of the posterior, $\mathbb{E}_{p_{1|t}}[z]$ (via Tweedie’s formula). Trajectories for which $r(\mathbb{E}_{p_{1|t}}[z])$ is low are removed, and trajectories with high scores are duplicated to replace them. This approach avoids backpropagating gradients of the reward function. However, it can fail due to computing the reward of the posterior mean, rather than a true sample from the posterior. Furthermore, it is expensive to simulate many trajectories at once.

3.2.1 Estimating the Value Function with Flow Maps

In the previous section, we discussed the methods to approximate $\nabla_{x_t} V_t(x_t)$

- Tweedie: Requires one cheap forward pass, but returns the posterior mean $\mathbb{E}[z \mid x_t]$
- Monte Carlo: Returns clean samples from the posterior $z \sim p_{1|t}(z|x_t)$, but each sample requires several forward passes to compute

We see a clear tradeoff between computing $\nabla_{x_t} V_t(x_t)$ cheaply and accurately. To obtain the best of both worlds, ideally we would like to sample a clean point $z \sim p_{1|t}(\cdot|x_t)$ (as in Monte Carlo), but only with a single differentiable forward pass.

A flow map allows for exactly this. $F(x_t, t, 1)$ jumps, in a single forward pass, from the current noisy point x_t straight to the clean sample at the end of the path at $t = 1$. So rather than using the posterior mean, $\mathbb{E}[z \mid x_t]$, we can use an exact clean sample of the posterior, in order to estimate the value function: $V_t(x_t) \approx r(F(x_t, t, 0))$. Why is this better approximation than $r(\mathbb{E}[z \mid x_t])$? Remember that the reward function r can only be meaningfully evaluated on clean data. The posterior mean, $\mathbb{E}[z \mid x_t]$, by contrast is often not a representative of a clean sample from the true data distribution, and is instead a blurry interpolation between the modes of the data distribution. This is especially the case for small t , and so the approximated $\nabla_{x_t} V_t(x_t)$ will give a poor steering signal early on in the trajectory. On the other hand, $F(x_t, t, 1)$ lands at a clean sample for any t .

Flow maps still have one major problem in the context of approximating the value function. The true value function V_t is computed as an expectation of the reward, r , over the entire posterior, $p_{1|t}(z|x_t)$:

$$V_t(x_t) = \log \mathbb{E}_{z \sim p_{1|t}(\cdot|x_t)}[\exp(r(z))] \quad (3.5)$$

However, since flow maps are deterministic, they can only produce a single sample from the posterior. Hence, flow maps produce a biased estimate of the value function $V_t(x_t) \approx r(F(x_t, t, 0))$. Computing an unbiased estimator (via Monte Carlo) requires obtaining many samples from the posterior $p_{1|t}(\cdot | x_t)$. To overcome this, stochastic flow map methods have been introduced.

Stochastic flow maps take the deterministic flow map $F(x_t, t, 1) = z$ and augment it with a noise variable ω so that the map becomes $F(x_t, t, 1, \omega) = z(\omega)$. The stochastic flow map is constructed such that varying ω at fixed x_t produces samples from the posterior

$$z(\omega) \sim p_{1|t}(\cdot | x_t) \quad \text{with} \quad \omega \sim \mathcal{N}(0, I).$$

Stochastic flow maps hence learn to map a tractable noise distribution onto the posterior, $p_{1|t}(\cdot | x_t)$. This allows us to produce many samples from the posterior, using one forward pass per sample, enabling cheap and accurate estimation of $\nabla_{x_t} V_t(x_t)$. Meta Flow Maps and Diamond maps are two different approaches to constructing a stochastic flow map, $F(x_t, t, 1, \omega)$.

Summary Methods for sampling from a reward-tilted distribution which are training-free with respect to an unconditional generator, all require guiding a sampling trajectory. Hence reward-alignment is many step. Methods such as GFT which train a flow map to learn trajectories following a guided probability path, enable one or few-step generation of samples from a tilted reward distribution. However, such methods require either fine tuning or training a flow map or model from scratch. To our awareness, there is no reward alignment method which is both training-free with respect to an unconditional generator, and enables one-step generation. This is the gap that our project aims to tackle.

Chapter 4

Method

4.1 The noise adapter as a normalising flow

Recall that the focus of this project is inverse problem solving, i.e. assuming a particular forward operator, and given any observation y^* , we want to sample from $p_{data}(x|y^*)$. VFMAimstosolvethisinthenoisespaceofanunconditionalflowmap.Inparticular,VF tuningtheflowmapparameterstheta,istoessentiallywarpthedistirbution $p_{theta}(z|y)$ suchthatitissi

This approach yields good performance for inverse problem solving, however jointly training the flow map can be very expensive. The natural solution is to simply make the variational distribution $q_{phi}(z|y)$ more expressive so that it can accurately approximate $p_{theta}(z|y)$ training with the flow map. In particular, we would like to choose a variational family which is more expressive than q_{phi} . This objective requires both being able to evaluate the log density of the variational distribution.

One possible choice is to choose $q_{phi}(z|y)$ as a Gaussian with full covariance matrix. However, given high dimensional problems, where K may need to be very large in order to approximate $p_{theta}(x|y^*)$.

Instead, we decided to approximate $q_{theta}(z|y)$ using a conditional normalising flow. Fix, an observation y^* . Recall that an (unconditional) normalising flow models $q_{theta}(z|y^*)$ by pushing a simple based distribution

4.2 Training objectives

4.2.1 Reverse KL

We have already shown that in order to approximate $p_{theta}(z|y)$ with a variational distribution $q_{phi}(z|y)$ based objective ?? . Recall that to derive this (see section on VI), we start with reverse KL between variational

$$\mathcal{L}(\phi) = D_{\text{KL}}[p_{\phi} \parallel p_{\text{target}}] = \mathbb{E}_{x \sim p_{\phi}} [\log p_{\phi}(x) - \log p_{\text{target}}(x)] \quad (4.1)$$

4.2.2 Forward KL

Recall that in the section on normalising flows, we stated that it is possible to train normalising flow by via using both reverse and forward kl divergence with the target distribution, as the training objective. We have already discussed objective derived from reverse kl divergence. However, it is in fact also possible to train via a forward kl objective. Recall that training via forward kl divergence reduces to maximum likelihood estimation. i.e. we wish to maximise the density that our model distribution $p_\theta(z|y)$ places at observed data z . Now, for a single observation y , we can evaluate $p_\theta(z|y)$. However, in our case we are amortising over the observation y , i.e. we want to learn $p_\theta(z|y)$ for any y . $\sim \mathcal{N}(0, I)$ 2) Decode through the frozen flow map to get clean data: $x = f_\theta(z)$ 3) Apply the forward operator and observation noise: $y = Ax + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2)$. By definition, every (z, y) pair generated this way is an exact sample from the generative model's joint distribution $p_\theta(z, y)$. Hence we can use the following maximum likelihood objective

$$\sum_{i=1}^N \log q_\phi(z_i; y_i)$$

where $(z_i, y_i) \sim p_\theta(z, y)$ for $i = 1, \dots, N$. Importantly, since q_ϕ is a normalising flow, we can evaluate

Note that for general reward alignment, forward kl is not a tractable training objective. inverse problem solving is a special case in which we can tractably obtain a sample z from the tilted target noise distribution, and only one such sample is sufficient because we are amortising over a family of target noise distributions.

4.2.3 Mixed Objective

Notoriously, forward kl is mode covering. In particular, the forward kl objective heavily penalises not placing mass at modes of the target distribution. However, placing mass where the target does not have mass, is not penalised. Therefore, a normalising flow is encouraged to place mass at all modes of the target distribution, often at the expense of placing mass between modes, where the true target has no density.

On the other hand, reverse kl is mode seeking. In particular, the reverse kl objective heavily penalises placing mass where the target distribution has none. Hence to maximise this objective, the normalising flow typically learns to place its mass at a single mode. A normalising flow has to 'tear' the Gaussian base

distribution in order to avoid placing mass between separate modes of a target distribution. This requires very large log det, which causes unstable training. And so a normalising flow typically learns to stick to a single mode.

To overcome the limitations of both, Frank Noé et al. propose using an objective combining both losses. In particular, they use a weighted sum of both loss function, where the weighting of the reverse kl loss progressively increases during training, according to a fixed schedule.

$$\mathcal{L} = \mathcal{L}_{\text{fKL}} + w(t) \cdot \mathcal{L}_{\text{rKL}}$$

To evaluate the fkl term, we must sample noise z , pass through the flow map to generate x , and construct the observation y . We can use the generated y (as opposed to observations y constructed from real dataset images) to evaluate the rkl term. This ensures that the direction that the fkl and rkl gradient pulls the parameters ϕ towards are aligned, and we hence expect less noisy overall gradient direction.

We now discuss how we determine an appropriate weight schedule, $w(t)$. Intuitively, the weight schedule should be such that the rkl term has weight 0 at the beginning. This is so that fkl objective can ensure that all modes of the training distribution are covered, even if mass is incorrectly placed between modes. We can then progressively increase the rkl weight. The rkl term can then sharpen the mass on the nodes, removing any mass between the found nodes, because incorrectly placed mass is exactly what rkl penalises. As previously discussed, a normalising flow may find it difficult to completely eliminate mass between modes, and so we still want to keep the forward kl, to avoid mode collapse. We think ending at 50/50 share between both terms is sensible.

After training with weight 0 for rkl term (i.e. fkl only), we compute the fraction of the total gradient contributed by the reverse-KL term $R = \mathbb{E}_{g_r} \text{kl} \mathbb{E} / (\mathbb{E}_{g_f} \text{kl} \mathbb{E} + w \mathbb{E}_{g_r} \text{kl} \mathbb{E})$ e.g. $\mathbb{E}_{g_f} \text{kl} \mathbb{E} = 105$, $\mathbb{E}_{g_r} \text{kl} \mathbb{E} = 9727$, so $R = 93$; we then pick w such that the gradient share prog

To determine what fraction total gradient the rkl term should take up, we try different weight ramps. In practice, we keep the same weight ramp, and we add a coefficient on the rkl term. This way there is only one hyperparameter, and the only thing that changes is not the steepness of the ramp, but the scale of the ramp. This is beta-VAE.

4.2.4 Noise Augmentation

We use noise augmentation. This section should discuss how the noise augmentation affects the loss functions.

4.3 Building the noise adapter

Our method can be summarised as training a conditional normalising flow $q_{\phi}(z|y)$ to approximate the distribution over noise space, $p_{\theta}(z|y)$. We have discussed that we can train on high-dimensional data, and we progress to experiments on imaged data. We separately construct the function

4.3.1 Low-dimensional Toy Problems

We first consider building a normalising flow for inverse problem solving in a 2D setting, with a 1D observation y . In particular wish to learn a diffeomorphism $f_{\phi}(z|y) : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, conditioned on y , which transforms samples from a base distribution, which we choose to be a standard Gaussian, into samples from the target distribution. We construct 4 layers, each layer being an identical autoregressive flow. Let us consider one such layer, which takes as input a 2D vector z and a 1D observation y . Each part z_i is transformed via an affine function $z'_i = \exp(a_i) * z_i + b_i$; the parameters of the affine function are computed as a function of $z_{<i}$ and the observation $c(z_{<i}, y)$. This is exactly the mechanism by which we learn a conditional normalising flow – the conditioner, c , takes as input the observation y , by simply concatenating it with z . The conditioner is implemented as a neural network with D output neurons for $z'_{\geq i}$. Severing connection weights can be implemented by multiplying the weight matrices of the MLP element-wise with a binary mask. Determining which weights to sever is simple if you only have one hidden layer. However, as hidden layers increase, this becomes difficult. MADE severs connection weights by assigning a 'degree' to each neuron in the MLP. Concretely,

1. Each input z_i is assigned with degree i
2. Every hidden neuron is randomly assigned a degree between 1 and $D - 1$
3. A neuron is connected to a neuron in the subsequent hidden layer only if the receiving neuron has equal or higher degree
4. The final output neuron for h_i can only be connected to hidden neurons in the previous layer which have degree strictly less than i

The conditioning input y gets the smallest degree (of 1), so that it is fully connected to all neurons in the first hidden layer. A final implementation detail is that we reverse the coordinates of z between each layer to ensure that the

transformation of each coordinate can be conditioned on the other coordinate. We implement the normalising flow using zuko 1.6.0.

4.3.2 Image-scale Problems

For all experiments at image-scale, we use the TarFlow architecture as the normalising flow. We use TarFlow as it is a state-of-the-art architecture for modelling image distributions. While we are not modelling a distribution over images directly, we are modelling a distribution over noised latents which are image-shaped. We discussed exactly the architecture of TarFlow in the background section, and we give the hyperparameters we choose in the experiments section.

Natively, TarFlow implements the conditioner of each autoregressive layer as taking as input z rather than z' . This means that computing f_p can be cheaply evaluated while f_p^{-1} is more expensive.

To make Tarflow conditioned on y , there are many possible approaches. Because in our experiments, the observation y is of varying dimensionality, we give our overarching approach, and provide specific details in the experiment section. Ultimately, we want the learned diffeomorphism to be parameterised by the observation y . TarFlow, by default, allows for class-conditioning by adding, at each layer, a learned class-specific embedding to each token once it has been projected to the internal dimension of the conditioner network, and the positional embedding has been added. Hence the conditioner’s output (the affine transformation parameters) depend on the observation y . Now because we do not have a fixed number of classes, we cannot use a simple learned embedding per-class. But we can train an encoder network, which given an observation as input, produces an embedding. And this embedding can be directly added to each token in the same way.

To increase the influence of the observation y , we can let the encoder produce both a shift and a scale, by which to transform the tokens (rather than solely a shift). This shift and scale can be made to be both token-dependent, and layer dependent, to increase expressivity. We can further increase the influence of the observation y by transforming tokens, not solely once per layer, but before and after each transformer block in each layer, giving $\text{LPB}+1$ total injections of the observation, per layer.

There are several other methods which can be used to inject information about the observation y into the normalising flow. For example Glow uses an affine injector. In particular, we can compute shift and scale of affine transformation of

z using solely the observation y. This is invertible assuming that we have y. We find in practice that as long as the observation encoder is sufficiently expressive, how the encoding information is delivered to the flow does not make a significant difference. Furthermore, while it is possible to pre-train the observaion encoder, we jointly train its prameters with the normalising flow, optimising the same forward or reverse kl objective.

Final note: How we ensure that the NF initialises as the identity?

4.4 Evaluation Framework

Ultimately, the goal of inverse problem solving is, assuming a particular forward problem which acts on data $x \sim p_{data}$, and given an observation y^* , produces samples from $p_{data}(x|y^*)$. More precisely, the goal of inverse problem solving is to sample from $p_{data}(x|y^*)$. Hence the ultimate

Note that in the framework of variational flow maps, we have two separate comoponents which work together to produce samples from $p_{data}(x|y^*)$: the noise adapter and the flow map. When using a reverse kl training objective (and keeping the flow map

Hence we can isolate the error of our noise adapter by comparing $q_{\phi, \theta}(x|y^*)$ with $p_{\theta}(x|y^*)$.

To summarise we have (1) $q_{\phi, \theta}(x|y) \rightarrow$ (2) $p_{\theta}(x|y) \rightarrow$ (3) $p_{data}(x|y)$. First gap is adapter error. Second gap is flow map error. We can compute distance between any two distributions e.g. between (1) and (2) using a metric such as MMD, which assumes we have samples from both distributions.

Maximum mean discrepancy (MMD) Given J samples $x^{(j)}$ from the first distribution and K samples $x^{(k)}$ from the second, we compute an (unbiased estimator) of MMD between the two distributions as

$$\widehat{\text{MMD}}^2 = \frac{1}{N(N-1)} \sum_{i \neq i'} k(x^{(i)}, x^{(i')}) + \frac{1}{M(M-1)} \sum_{j \neq j'} k(y^{(j)}, y^{(j')}) - \frac{2}{NM} \sum_{i,j} k(x^{(i)}, y^{(j)}) \quad (4.2)$$

where we choose a Gaussian kernel, $k(a, b) = \exp(-\|a - b\|^2 / 2\sigma^2)$. A kernel $k(a, b)$ is a similarity measure. In the case of a Gaussian kernel, $k(a, b)$ is 1 when a and b are exactly equal and falls to 0 as they grow further apart. Here, σ is the lengthscale hyperparameter. We set it using the median heuristic, which simply sets it to the median of pairwise distances between the $K + J$ samples in the pooled set formed by combining both sets. We make sure to use the same σ to evaluate MMD for all methods being evaluated in a single setting.

The first two terms of ?? correspond to the 'within-set' similarities computed

separately for each distribution, while the third term corresponds to the ‘cross-set’ similarity between the two distributions. Intuitively, if the two sample sets come from the same distribution, the within-set similarities and the cross-set similarity balance and MMD will roughly equal 0. If they differ, the cross term is small while the ‘within-set’ terms stay large, so MMD will be large.

Can we compute MMD between any pair of distributions out of (1), (2) and (3) in practice? Of course our method gives us samples from (1). For simple low-dimensional experiments, rejection sampling can often be used to obtain samples from both (2) or (3). In particular, sample x from $p_{t\theta^*}$, accept with probability $p(y^*|x)$. By construction, we will obtain samples distributed according to $p_{t\theta^*}(x|y^*)$. We scale experiments, the acceptance probability will be so low that it is computationally in

Therefore, we must consider alternative metrics for image-scale experiments, which indirectly give evidence of whether the inverse problem solving method is successfully producing samples from $p_{data}(x|y^*)$.

We now discuss such metrics. We separate the metrics we use for low-dimensional experiments, and the metrics we use for image scale experiments.

Additional metrics for Low-dimensional experiments

Given an observation y in R, the output of all inverse problem solving methods that we evaluate (both ours and baseline methods), are N samples $(x_i)_i = 1^N$ from a distribution approximating $p_{data}(x|y)$. For VFM, such N samples are obtained by sampling from $q_p \text{hi}(z|y)$, and denoising via the flow map $x_i = f_t \text{theta}(z_i)$. We determine the quality of the N samples produced by our method, for a particular observation y , via the following metrics. While we state each metric for a single measurement $Ax_t^{true} + e$, where $e \sim N(0, \sigma^2)$ For all metrics, we provide the formula of its corresponding tractable approximation.

Negative log predictive density (NLPD) For a single observation y , and samples $(x_i)_i = 1^N$, *NLPDiscomputed as* $NLPD(y) = -\log \left[\frac{1}{J} \sum_{j=1}^J \mathcal{N}(y; Ax^{(j)}, \sigma^2) \right]$ (4.3) where σ is the standard deviation of the gaussian noise in the forward model. Ignoring the normalising constant of the Gaussian, and the negative log transformation, this metric averages $|Ax_i - y| / \text{sigma}$, overall x_i . i.e. it computes the average euclidean distance of ft

Continuous ranked probability score (CRPS) Given a ground truth target, x_{true} , and $samples(x_i)_i = 1^N$, $CRPS$ is computed as $CRPS(y) = \frac{1}{J} \sum_{j=1}^J \|x^{(j)} - x_{true}\| - \frac{1}{2J^2} \sum_{j=1}^J \sum_{k=1}^J \|x^{(j)} - x^{(k)}\|$ (4.4) The first term in ?? computes the average euclidean distance between each of the N samples to the ground truth target x_{true} . The second term computes the averaged distance between any two of the N samples. $CRPS$ is minim

Support accuracy (SACC) Given samples $(\mathbf{x}_i)_{i=1}^N$, *SACC* computes the proportion of samples

dimensional experiment, $p_{data}(x)$ is such that given a point x in R^2 , either $p_{data}(x) = 0$ or $p_{data}(x) = c$, for some constant c . Hence we can compute exactly the proportion of samples which are

Image-scale experiments

Following VFM, for image-scale experiments we use metrics: PSNR, SSIM, LPIPS, FID, MMD, CRPS. We now give an overview of each metric, noting that the metrics MMD, CRPS are used differently compared to for the low-dimensional experiments.

PSNR Consider a single observation y , produced by applying the forward model to a sample x_{true} p_{data} . Additionally for a given inverse problem solving method, which aims to solve $y = Mx$, we compute the PSNR as $10 \log_{10} \left(\frac{MAX^2}{MSE(\hat{x}, x_{true})} \right)$ where $MSE(\hat{x}, x_{true})$ is standard mean squared error in pixel-space between across all $D=H \times W \times 3$ dimensions, and MAX is the range of the values a pixel can take. If images are in $[-1, 1]$, we use $MAX=2$. Intuitively, PSNR evaluates, in pixel-space, the fidelity of a single output sample to the true sample x_{true} , which produced the observation y . The PSNR score is normalised against the range of possible pixel values, so that frequency details are lost. Hence this must be taken account when interpreting PSNR scores.

SSIM SSIM also evaluates the fidelity of a single sample x to x_{true} . To evaluate the fidelity of x , we compare x and x_{true} on overlapping patches. For each patch of x , we compute the similarity to the corresponding patch of x_{true} as

LPIPS Again, LPIPS evaluates the similarity of a single sample x to x_{true} . However, rather than comparing the images in pixel-space, as the previous two metrics do, LPIPS compares the similarity of the L dimensional feature vectors at each spatial allocation. LPIPS chooses L layers for which we compare the feature vectors. For each layer, and each spatial allocation in that layer, we normalise the feature vector to unit length, and then compute the L2 distance between the two vectors.

FID (Fréchet Inception Distance) All metrics so far evaluate how close a single produced sample is to the corresponding ground truth sample, x_{true} , and average over the test set. On the other hand, we use FID to evaluate how similar a whole collection of samples is to the ground truth.

We now give more detail on how exactly FID evaluates the similarity between two collections of samples. Each image in both collections is passed through an Inception-v3 network, and the activations of the final pooling layer are taken, giving a feature vector per image. To evaluate the distance between the two sets of image features, FID approximates each set as being distributed according to a Gaussian. For each set, FID computes the mean vector and covariance matrix. FID then computes the Fréchet distance (also called the 2-Wasserstein distance) between the two Gaussians. FID is zero when the two Gaussians coincide, and larger as they differ.

We use a sample size of 50k for each set, in order to evaluate FID. Additionally, because Inception-v3 expects 299x299 inputs with input pixel range of $[0, 255]$, we upsample our images which are 32x32, and scale our pixel values to this range.

MMD) We compute MMD, using the exact sample unbiased estimator ?? we previously saw, but now between the same two sets for which FID is evaluated on. Just like for FID, each image in both sets is passed through an Inception-v3 network, and MMD is computed between the network features of each set. We again set the lengthscale σ using the median heuristic. In particular, we use our samples from our method to compute σ , and use the same σ to evaluate MMD for all other methods. Unlike FID, MMD does not make any assumption about the distribution of both sets. Thus even if the two sets have moments which are equal, while FID would report a good score, MMD may not.

CRPS We use the same estimator for CRPS as we used in low-dimensional experiments ??. Rather than evaluating CRPS in pixel space, both the ground truth sample $\mathbf{x}_{true}$, and samples generated from the inverse problems solving method are converted to feature space using an Inception-v3 network. We again use the feature space of a network so that the pairwise distances in ?? capture perceptual differences. In scale experiments, CRPS is the only metric which is minimised when the predictive distribution of the model is closest to the ground truth. We also choose $K = 128$ observations to average over.

Chapter 5

Experiments and Results

5.1 2D Checkerboard Example

Before scaling our method of using a Normalising Flow adapter with a frozen flow map to image-level experiments, we begin by illustrating our method on a toy 2D example. In this 2D setting, we compare our method to both a Gaussian adapter with a frozen flow map and with a jointly trained flow map (VFM). The aim of this experiment is to show that, in a simple 2D setting, a Normalising Flow adapter can successfully learn the inverse problem posterior without requiring fine-tuning of the flow map, whereas a simple Gaussian adapter fails. While the authors of VFM only train the Gaussian adapter with frozen flow map using a reverse kl objective, we additionally train with forward kl to rule out the possibility that the mode seeking failure they observe is due to the reverse kl objective.

5.1.1 The Data Distribution

In our 2D experiments, we take $p_{data}(x)$ to be a 4×4 checkerboard placed on a support of $[-2, 2]$. We generate `n_samples` points uniformly within the unit square $[0, 1]^2$.

Divide that unit square into a 4×4 grid, giving 16 cells (each 0.25 wide and tall)

Assign each of the `n_samples` points with a pair of indices (i, j) (each from 0 to 3) corresponding to which cell that point belongs to

Keep a point only if $i + j$ is even. This will ensure we retain points which form a checkerboard pattern.

For every discarded point, sample a new point until we obtain `n_samples` valid points

Shift all `n_samples` points by $(-0.5, -0.5)$ and scale by 4 such that the checkerboard fills a 4×4 region centered on the origin $([-2, 2]^2)$.

5.1.2 Defining the Inverse Problem

We define the following forward problem. Given $x \sim p_{data}(x)$, we compute an observation $y = Ax + e$, where $A = [0, 1]$ and $e \sim N(0, \sigma^2)$, where we take $\sigma = 0.1$ in our experiments, follow $\exp\left(-\frac{|y-Ax|^2}{2\sigma^2}\right) p_{data}(x)$. In words, given a measurement corresponding to the noised first coordinate of a sample from p_{data} , produces samples from p_{data} which also have the same y . The resulting distribution has either 2 or 4 disconnected modes.

5.1.3 Base Flow Map Training

For all methods which we evaluate, whether the method keeps the flow map frozen or fine tunes it, requires a base flow map which can generate samples from the unconditional checkerboard data distribution, $p_{data}(x)$. We use a flow map with model weights trained by VFM authors. In particular, they train it to predict the instantaneous velocity using a flow matching training algorithm, using a CondOT initialised optimiser.

5.1.4 Noise Adapter Training

We now briefly discuss how we train the noise adapter in our method alongside the baseline methods.

Normalising Flow Adapter

We use the normalising flow architecture as outlined in ???. We separately train the normalising flow noise adapter using forward kl and reverse kl objectives. During training, we always perform one-step prediction using the flow map, i.e. $K=1$. Specifically, when training using forward kl, we generate samples using the flow map with $K=1$ decode steps, and when decoding the outputs of the noise adapter when evaluating the reverse kl loss, we also use $K=1$ decode steps. Both

objectives use batch 2048 and gradient-norm clipping at 5.0. When training via forward kl, we use 5000 iterations, Adam, lr 1×10^3 . Each iteration draws a fresh batch of samples from the flow map, so there is no fixed training set. On the other hand, reverse kl training uses 20,000 iterations, AdamW, lr 2×10 , weight decay 1×10 . We use the same fixed set of 20,000 true checkerboard samples that we used to train the flow map, to train the adapter using reverse kl. Because forward KL is a plain regression on exact pairs, training is more stable and so we can use a higher learning rate with fewer steps.

Gaussian Adapter with frozen θ

Following the VFM authors, the Gaussian noise adapter is a SiLU MLP with four hidden layers of width 256 taking the scalar observation y as input with two linear heads outputting (y) in $\mathbb{R}^2$ and $\log(y)$ in $\mathbb{R}^2$ (clamped to $[-6, 3]$ for stability). We train using forward kl

VFM

We again follow exactly the same training procedure as the VFM authors for joint training of the noise adapter and flow map. The noise adapter is the same diagonal-Gaussian architecture as in the frozen θ baseline. The flow map is initialised from the flow-matching stage checkpoint (not the MeanFlow model). We additionally maintain a stop-gradient EMA copy θ of the flow map with decay = 0.999, updated every step. We use the training loss ?? with = 100. In the observation loss term of ?? is computed by decoding the adapter’s noise sample via the EMA copy of the flow map. Hence the gradient with respect to observation loss is only backpropagated to the noise adapter, not flow map. The MeanFlow term is computed using noise solely proposed from the noise adapter (rather than $N(0, I)$), rather than using samples from both with proportion α and $1 - \alpha$ respectively. Adapter and flow map are optimised jointly by a single AdamW (learning rate 2×10 , weight decay 1×10) for 50,000 iterations at batch size 2048, drawn from the same fixed set of 20,000 checkerboard samples. For evaluation, we use the raw trained flow map rather than the EMA copy.

5.1.5 Results

full page figure true $p(y | x)$ Gaussian strawman rkl Gaussian strawman fkl NF adapter rkl NF adapter fkl VFM

6 rows, 2 columns. first column corresponds to data space, second column corresponds to noise space. for $p(y | x)$, second column for noise space is empty all using $K=4$.

Table with columns Method, NLPD, CRPS, SACC, MMD between (1) and (2), MMD between (2) and (3), MMD between (1) and (3)

5.2 Image Inverse Problems

5.2.1 Common Setup

Base Flow Map

Evaluating the Flow Map

TarFlow Noise Adapter Hyerparameters

For brightness: tokenisation - 4x4 patches, which gives 64 tokens, each of 48 dims number of blocks - NB=4 layers per block - LPB = 2 causal self-attention layers residual stream dimension - CH = 256 attention head info: 4 heads, each of internal dimension 64. head outputs are concatenated, before being passed through Linear(256 $\rightarrow$ 256) MLP expansion - 4, i.e. MLP expands 256 residual stream into 256x4 6.6M parameters in all. (no conditioning)

Optimiser: Adam (0.9, 0.999), LR 5e-4 cosine $\rightarrow$ 1e-5, grad-clip 5.0,

PyTorch 2.13/cu130, single RTX A5000

5.2.2 Brightness Steering

The forward problem: $y = Ax + e$. Here, $A: \mathbb{R}^D \rightarrow \mathbb{R}$, computes the mean of all $3 \times 32 \times 32$ pixel values. These pixel values are in $[-1, 1]$ for CIFAR10. We choose to be Gaussian with mean 0 and $\sigma = 0.05$. σ is chosen to be small enough so that samples from $p(x|y)$ are visibly different from $p(x)$, but still

The inverse problem is then, given measurement y^* , determine samples from $p(x | y^*)$. For this task, we fix $y^*=0.2$. We chose this value of y^* because it is roughly one standard deviation above the mean brightness level (of 4000 generated samples from the flow map).

We wanted to progressively increase the difficulty of the inverse problems that we choose. In practice, this means choosing inverse problems with progressively larger dimension measurement. Hence we started with an inverse problem with a 1D measurement. We specifically chose brightness as the measurement because it allows us to visually determine the effectiveness of the noise adapters that we test.

Aside from using this experiment as evidence that the normalising flow adapter works for image-level problems, so that we can move on to harder image inverse problems, the point of this experiment is to give evidence that a Gaussian adapter is insufficient. This is why we did not do amortisation here. We want to isolate the failure of a Gaussian adapter to the lack of expressivity of the Gaussian distribution, rather than any failure due to amortisation, often referred to as the Amortisation gap in the literature.

Additionally, rejection sampling is computationally feasible, and so this enables more rigorous evaluation (we can directly compare with ground truth samples from $p_{data}(x|y^*)$) before moving on to harder inverse problems, where we can't do such rigorous evaluation.

Gaussian Adapter Training

We use diagonal gaussian, and we directly optimise its mean and variance. While it is possible to use full covariance gaussian, it would be computationally infesible when moving to the amortised setting. In particular, for $D=3072$, the covariance matrix would have 4.72M entries. An output layer that maps an e.g. 512 dimension feature vector to 4.72M values, would require 2.4 billion parameters. The original VFM authors also use diagonal gaussian for their noise adapter. As we we are only trying to approximate a single distribution $p(x | y^*)$, there is no need to train a neural network as for the amortised case. log clamped to $[6, 3]$; initialised at $\mu = 0$, $\log \sigma = 0$.

We optimise the mean and variance in two ways: maximum likelihood, and reverse KL.

In maximum likelihood, we have a dataset of observations from the true $p(y^* | x)$. We can then compute the maximum likelihood estimate of the mean and variance in closed form. no need for optimisation. The only problem with this is that to obtain a good maximum likelihood estimate, we require a very large number of observations.

For reverse KL, the objective is what we already know. We can compute the KL between the gaussian and the prior standard normal exactly (no need for sample based estimator).

We update mean and variance parameters using gradient descent. We can use Adam, learning rate 2e-4, cosine-annealed $\rightarrow$ 1e-5 over the run. (this was tuned), batch 128 for 6000 steps. gradient clip of 5.

Results: rkl diag is bad. fkl diag is quite good.

Noise Adapter Training details

6000 steps Batch 64

NF vs Gaussian Adapter Ablation

5.2.3 Super-Resolution

The forward problem: $y = Ax + e$. x is $[-1,1]^{3 \times 32 \times 32}$ Here, A represents bicubic downsampling. $32 \times 32 \times 3 \rightarrow 8 \times 8 \times 3$. We choose to be Gaussian with mean 0 and $\sigma = 0.05$. validation set is 1000 real CIFAR-10 test images

Noise Adapter Training details

Conditioning mechanism How do we inject information about y into the TarFlow adapter? We first process the observation y using a CNN. CNN consists of three 3×3 convolutions: $\text{Conv}(3 \rightarrow 64) \rightarrow \text{SiLU} \rightarrow \text{Conv}(64 \rightarrow 64) \rightarrow \text{SiLU} \rightarrow \text{Conv}(64 \rightarrow 64)$ The spatial resolution remains 8×8 after each convolution So that the output has dimension $(64, 8, 8)$ We now have a 64-dimensional feature vector for each of the $8^2 = 64$ spatial positions/pixels in the observation. Recall that TarFlow splits z into a grid of 8×8 tokens. $z_{i,j}$ corresponds to one spatial correspondence between each of the 64 feature vectors of the observation, and the tokens.

For each of the NB=4 blocks, we inject the observation feature vectors into the normalising flow. For each block, we have a separate learned linear layer $\text{Linear}(64 \rightarrow 256)$, which maps the 64 observation feature vectors to the internal dimension of the conditioner neural network. For each token embedding, immediately after $\text{proj}_i + \text{positionalemb}$, we add the projected observation feature vector corresponding to the same spatial position. Conditioner 0.14M parameters; whole model 6.67M.

rKL vs fKL vs Mixed Objective Ablation

All use cosine schedule $5e-4 \rightarrow 1e-5$ for LR. differ in number of steps: fkl 12k, mix 12k, rkl 6k fkl batch 128 mix: batch 128 (fkl) + 32 (rkl term, first 32 of the same y batch) rkl: 32 x 2 accumulation (eff. 64) wall clock is 80 min fkl, 210 min mix, 279 min rkl.

weight ramp for mix is piecewise constant: $5.5e-4$, $2e-3$, $7.2e-3$, $1.3e-2$ 0 until step 6k, then weight ramp applied at steps 6000 / 7000 / 8500 / 10000

for weight ramp: during training, we found that the realised share stayed within roughly within roughly ± 10 points of target throughout the ramp (6.5% at the start; 44-59% over the final stage),

Comparison with Baselines

5.2.4 Inpainting

Noise Adapter Training details

Conditioning mechanism

Conditioning mechanism Ablation

β -VAE Ablation

Comparison with Baselines

Chapter 6

Conclusion

6.1 Summary

6.2 Future Work

6.3 Critical Evaluation

Chapter 7

Supplementary Material

7.1 Change of Variables

7.2 Score functions and score matching

7.3 Additional Experiments

7.3.1 GMM Experiments

Capacity Reward Hack in SR Inverse Problem

Appendix A

Supplementary Material

A.1 Change of Variables

A.2 Score functions and score matching

A.3 Additional Experiments

GMM experiment: does the method work when scaling to higher dimension?